Both have the DOCTYPE declaration, a heading for the company, and a copyright at the bottom. Such common markup can be factored out of the template files and placed in the layout document. Rails copies the markup in the layout document to all template documents in the application. Following is a layout document for the cars application:

```
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- base.rhtml - the layout document for the cars
                  application -->

<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Cars </title>
  </head>
  <body>
    <h1> Aidan's Used Car Lot </h1>
    <%= @content_for_layout %>
    <hr/>
    <p> Copyright 2007, AUCL, Inc. </p>
  </body>
</html>
```

This layout document is named base.rhtml, but there is nothing special about the name. This document is placed in the layouts subdirectory of the views subdirectory of the cars application.

Notice that the value of the predefined instance variable @content_for_layout is inserted in the body of the layout document. This is the way the document informs Rails of the position of the content of a particular template document should appear in the layout. So, every template or view document will be placed by this layout document, with the content of the particular template document inserted in the body of the layout document. This saves the developer the trouble of including the DOCTYPE declaration, the page title, and the copyright at the bottom in every template for the application.

Rails must be informed of the name of the layout document. This is done by placing a layout command in the ApplicationController class of the application. Recall that the controller directory of an application has at least two files, one named application and the others named by the developer. The application file has an empty class definition named ApplicationController. The form of the layouts command is just the word layouts and the name of the layout document, without the file name extension, in double

quotes. For example, to use `base.rhtml` in the `cars` application, the `ApplicationController` class for `cars` would be modified to appear as:

```
class ApplicationController < ActionController::Base
  layout "base"
end
```

If we add the `base.rhtml` document to the `layouts` directory and modify the `ApplicationController` class of the `cars` application, we would of course need to remove the `DOCTYPE` declaration, the page title, and the copyright from the two template files for that application, `welcome` and `result`.

Summary

Rails is a Ruby software framework for Web applications. Though it is applicable to all Web applications, it is particularly adept at Web applications that interact with relational databases. One characterizing aspect of Rails is its use of an object-relational mapping for connecting object-oriented Ruby to relational databases. Rails uses the Model-View-Controller model of software applications.

A Hello World Rails application can be easily built. A basic skeletal application is built with the `rails` command, giving the application's name as a parameter. This creates many directories and files that support the application. A controller class can be generated by running the `script/generate` script, providing a name for the controller as a parameter. Then an empty action method is added to the controller class. The last step of developing this application is to build the view, or template file, whose name must be the same as the action method in the controller. The template file in this case is a simple XHTML document whose content is `Hello World`. After starting a Web server, this application is ready to be requested by a browser.

Dynamic documents in Rails are closely related to those constructed with PHP. In this case, Ruby code is embedded in the template in the `<%` and `%>` brackets. When requested by a browser, the Ruby code is interpreted and its output is placed in the template, which is then returned to the requesting browser. In most cases, data and computations are placed in the controller action method. All instance variables in the action method are visible in the template file.

Form processing in Rails is relatively simple. Form values are available to the controller class through a hash-like object. The action method extracts the form values into instance variables.

Rails applications are cleanly integrated with database servers. MySQL is part of the InstantRails package, so it is especially convenient to use. After a database has been constructed, Rails is informed of the relationships among the tables with the directives, *belongs to* and *has_many*. The tables are accessible to the controller through classes whose names are singular forms of the table

names, with the first letter in uppercase. The rows of the tables are objects of the table classes. The items in a table row are available as fields of the table objects. The `find` method of the table classes provides a powerful way to extract data from the database.

Layouts provide a convenient way to include boilerplate markup in all of the templates of an application. Such boilerplate markup is placed in a template file in the `layouts` subdirectory of the `views` directory. Rails is informed of the layout template with the `layout` directive in the `ApplicationController` class.

Review Questions

15.1 For what is MVC an acronym?

15.2 For what is ORM an acronym?

15.3 What is generated with the `generate controller` script?

15.4 What must be placed in an application's controller class?

15.5 In what directory are templates placed?

15.6 Why does a template file's name have the `.rhtml` extension?

15.7 How are simple form data gotten by a form-processing action method?

15.8 In what directory are database files stored?

15.9 How are the model files for the tables of a MySQL database created?

15.10 What does the `belongs to` directive indicate to Rails?

15.11 What does the `has_many` directive indicate to Rails?

15.12 What does the `has_many :corvettes` directive in the `CorvettesEquipment` class cause Rails to do?

15.13 What class does Rails build for a table named `states` and what is in its objects?

15.14 What is a layout?

Exercises

15.1 Describe briefly an MVC application.

15.2 Describe briefly the ORM used by Rails.

15.3 Build a simple Rails application to return a static document to a requesting browser, where the static document is a brief description of you.

15.4 Build a Rails application that accepts two integer values and produces the product of the two values and returns it to the client.

15.5 Build a Rails application that constructs the `cars` database and accepts user requests for cars of a specific year, body style, and state and returns a list of such cars from the database.

CHAPTER

16

Introduction to Ajax

16.1 Overview of Ajax
16.2 The Basics of Ajax
16.3 Rails with Ajax
Summary ◦ *Review Questions* ◦ *Exercises*

This chapter provides a brief introduction to Ajax. As described in Chapter 1, "Fundamentals," Ajax is a process of using asynchronous requests from the browser to the server to update a part of the displayed document. The first section is an overview of the concepts and processes of Ajax. This is followed by a simple but complete example of Ajax being used to help a user fill a form. First, the application is described. Then, the initial form document is developed. Following this, the request phase, which creates the `XMLHttpRequest` object, attaches the user data, and sends it to the server. The example is completed with the development of the receiver function. Next, the issues of cross-browser implementation of Ajax are discussed. The following section describes the process of implementing Ajax within the Rails system. This includes developing the initial view document, describing the helper function `observer_field` to trigger the Ajax action, and developing the controller processes to implement the application.

16.1 Overview of Ajax

As discussed in Chapter 1, "Fundamentals," the goal of Ajax technology is to provide Web-based applications with responsiveness approaching that of desktop applications.

A typical session of Web use begins with the user requesting an initial document, either by typing a URL or clicking a link on his or her browser. At that point, the browser is blocked from activity while it waits for the server to provide a new document. When the document arrives, the browser replaces the former display with a rendering of the new document. This cycle takes some time, both in network latency and in rendering time. Nothing can be done to speed this process of fetching and rendering a complete document. However, user interactions with the displayed document may require that only relatively small parts of the displayed document are modified or updated. In a non-Ajax Web application, even the smallest change in the displayed document, if it needs data from the server, requires the same process that produced the initial display. The request must go to the server, the server must construct and send back a complete document, and the whole display must be re-rendered. All of this time, the browser is locked and the user can do nothing but wait. If a Web application requires many such interactions, the workflow of the user can be seriously disrupted.

Ajax is meant to significantly speed Web applications that have frequent user interactions. For those user requests that update only a small part of the displayed document, Ajax technology shortens the required time for both document transmission and document rendering. It does this by having the server provide only a relatively small part of the displayed document—the part that must change. This shortens transmission time, because the document being transmitted is much smaller, and the rendering time, because once again, only a small part of the display must be re-rendered. This is a simple idea, but one that can provide great improvements in the richness of the Web user experience, at least with applications that have frequent browser/server interactions.

Another key feature of Ajax is that those interactions between the browser and the server that result in small changes to the displayed document are asynchronous (the 'A' in Ajax). This means that when the browser requests a new part of its displayed document from the server, it does not need to lock while it waits for the response. The user can continue to do whatever he or she wants with the browser during the time it takes to fetch and render the new document part.

Traditional (non-Ajax) browser interactions with a server and Ajax interactions with a server are shown in Figure 16.1.

Ajax is not a new programming language or even a new API. In fact, one of the most attractive characteristics of Ajax is that it does not require Web programmers to learn new programming languages or markup languages in order to build Web sites that use Ajax. Ajax, true to its name, uses JavaScript as its primary programming language. Most Web programmers, certainly including

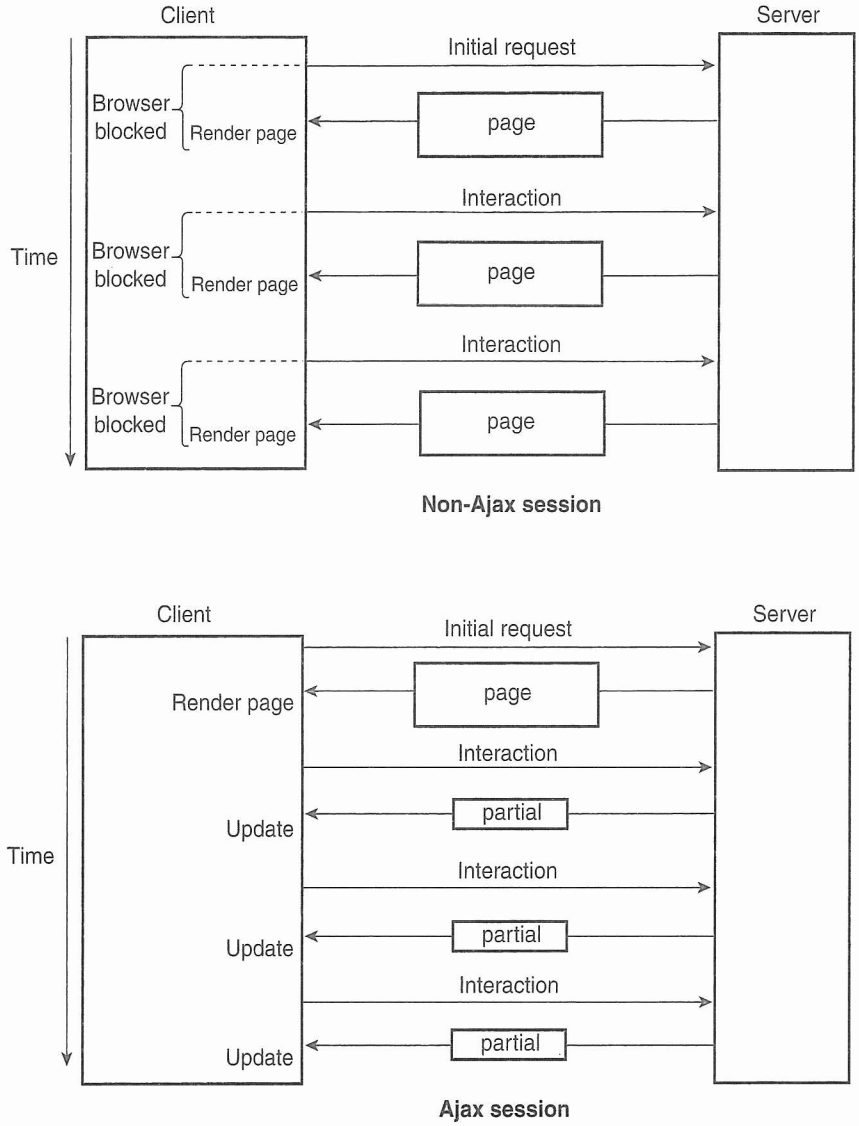


Figure 16.1 Traditional and Ajax browser/server interactions

those who have studied this book, already know JavaScript. The 'x' in Ajax represents XML. XML is used in Ajax, but not exclusively. Again, most Web programmers also already know XML. The other technologies used in Ajax are the DOM and CSS. These are also well-known to Web programmers (and readers of this book). So, Ajax is very attractive in the sense that no new technologies must be acquired or learned to use it. Furthermore, the technologies it uses are already present on the vast majority of Web browsers.

While Ajax uses JavaScript on the client side, it can work with virtually any server-side languages and technologies; for example, PHP, Java servlets, and ASP.NET. The return document can be XML, XHTML, JavaScript to be interpreted on the browser, or even plain text.

The first possibility of the Ajax approach arrived with the introduction of `iframe` element in the fourth versions of the browsers from Netscape and Microsoft. Web programmers discovered that an `iframe` element could be made to be invisible, simply by setting its width and height to zero pixels, and that it could be used to send asynchronous requests to the server. Although this worked, it was far from elegant.

Microsoft introduced two nonstandard extensions to the DOM and its JavaScript binding with the `XmlDocument` and `XMLHTML` objects, which began as ActiveX components in IE5. These were designed to support asynchronous requests to the server, thereby allowing data to be fetched from the server in the background. These are now supported by most commonly used browsers, although Microsoft's `XMLHTML` is named `XMLHttpRequest` in most browsers.

There were some developers using Ajax technology before 2005, but there was no widespread interest in it or enthusiasm for it. Two events were the catalysts that began the rush of Web developers to Ajax in 2005 and 2006. First, many users began to experience the rapid browser/server interactions provided by Google Maps and Gmail, which were some of the early Web applications to use Ajax. For example, Google Maps can quickly replace small parts of the displayed map, called tiles, using asynchronous requests to the server. Most had never used a Web application with such powerful interactive capabilities. Second, as mentioned in Chapter 1, "Fundamentals," Jesse James Garrett named this technology Ajax in early 2005. It may seem odd to some, but in this case, the acquisition of a name was an important part of the motivation for the huge growth in interest in the new approach to building Web applications.

16.2 The Basics of Ajax

In this section, a Web application is developed and used to describe the basics of Ajax.

16.2.1 The Application

The application used in this section, which has been used previously for the same purpose,[1] is as devoid of complexity as possible and still able to illustrate the fundamentals of the Ajax technology. The example initially displays the first part of the popcorn sales form used in Chapters 2, 11, and 15. Only the first part of the form, which gathers the name and address information from the user, is used. Just to make it more attractive, a small picture of popcorn has been added.

1. A similar example appears in *Ajax in Action*, D. Crane et al., Manning Publications, 2006, Greenwich, CT.

The concept of this application is that it uses Ajax to help the user enter his or her address information. Specifically, the form displays the text box for the user's zip code above the text boxes for the city and state of residence. When the zip code has been entered, signaled by the DOM as the `blur` event on the zip code's text box, a call is made to a JavaScript function that constructs an asynchronous request to the server. The zip code is sent to the server in the request, which uses it to look up the city and state, which are then returned to the browser. When the JavaScript code receives the city and state, it inserts them in the city and state text boxes in the form.

16.2.2 The Form Document

The first thing needed for this application is the document to present the form. One special requirement of this document is that the zip code text box must register a JavaScript function handler for its `blur` event. The call to the handler must pass the value of the zip code widget. This value can be referenced as `this.value`. The handler is named `getPlace`. Another requirement is that both the city and state text boxes must have id attributes, so that they can be addressed conveniently by the code that must insert the values returned from the server. Finally, the document must reference the JavaScript code file in a script element in its head. The complete document, named `popcornA.html`, follows:

```
<?xml version = "1.0" encoding = "utf-8" ?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- popcornA.html
     This describes popcorn sales form page which uses
     Ajax and the zip code to fill in the city and state
     of the customer's address
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Popcorn Sales Form (Ajax) </title>
    <style type = "text/css">
      img {position: absolute; left: 400px;  top: 50px;}
    </style>
    <script type = "text/JavaScript" src = "popcornA.js">
    </script>
  </head>
  <body>
    <h2> Welcome to Millenium Gymnastics Booster Club Popcorn
         Sales
    </h2>
    <form action = "">
```

```
<!-- A borderless table of text widgets for name and address -->
      <table>
        <tr>
          <td> Buyer's Name: </td>
          <td> <input type = "text"  name = "name"  size = "30" />
          </td>
        </tr>
        <tr>
          <td> Street Address: </td>
         <td> <input type = "text"  name = "street"  size = "30" />
          </td>
        </tr>
        <tr>
          <td> Zip code: </td>
          <td> <input type = "text"  name = "zip"  size = "10"
                      onblur = "getPlace(this.value)" />
          </td>
        </tr>
        <tr>
          <td> City </td>
          <td> <input type = "text"  name = "city"  id = "city"
                      size = "30" />
          </td>
        </tr>
        <tr>
          <td> State </td>
          <td> <input type = "text"  name = "state"  id = "state"
                      size = "30" />
          </td>
        </tr>
      </table>

      <img src = "popcorn.jpg"  alt = "picture of popcorn" />
      <p />

<!-- The submit and reset buttons -->
      <p>
        <input type = "submit"  value = "Submit Order" />
        <input type = "reset"  value = "Clear Order Form" />
      </p>
    </form>
  </body>
</html>
```

A display of the `popcornA.html` document is shown in Figure 16.2.

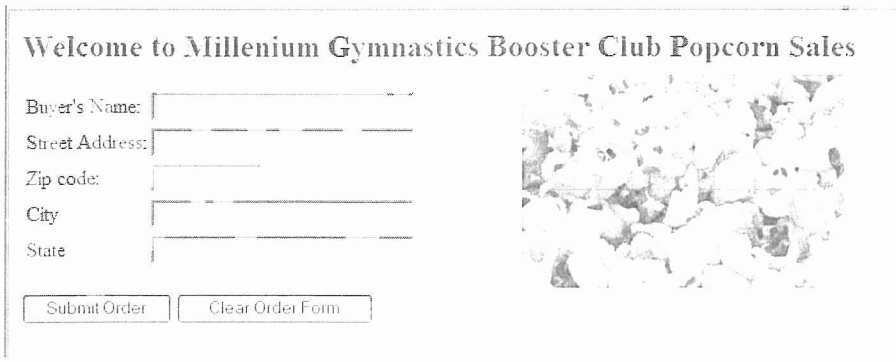


Figure 16.2 A display of the `popcornA.html` document

16.2.3 The Request Phase

The application requires two functions, the `blur` handler and a function to receive the response from the server. This section discussses the request phase—the `blur` handler. The receiver phase is discussed in Section 16.2.5.

The request phase of the application is entirely focused on the object used to communicate asynchronously with the server, `XMLHttpRequest`. The first step is to create an object using the `new` operator and calling the `XMLHttpRequest` constructor, as shown in the following:

```
var xhr = new XMLHttpRequest();
```

For the remainder of this chapter, we will refer to `XMLHttpRequest` as XHR. The XHR object has six properties and six methods. However, for now only two of the properties and two of the methods will be discussed.

When the server receives a request through an XHR object, it notifies the sender several times while it is servicing the request. These notifications are called *callbacks*. They are meant to inform the sender of the progress being made by the server on the request. There are five different values returned by the server to indicate progress, `0 .. 4`. The only one of these of interest is `4`, which indicates that the response is complete. This indicator will be used in the receiver phase. The receiver function, `receivePlace`, is the function called in the callbacks.

The next part of the request is to register the function that implements the receive phase of the application, which is named `receivePlace`. This function is registered to the `onreadystateChange` property of the XHR object, as follows:

```
xhr.onreadystateChange = receivePlace;
```

Note that this is not a call to `receivePlace`; it is an assignment of the address of the function to the `onreadystateChange` property. Therefore, there can be no parentheses following the name of the handler function. This handler registration causes `receivePlace` to be called several times while the server deals with the request, each time setting the `readyState` property of the XHR object to the progress value. Section 16.2.5 describes how the `receivePlace` handler deals with this.

The next step for the `getPlace` handler is to call the `open` method of the XHR object. The `open` method makes the necessary arrangements for the server request. This method takes two required parameters and three optional parameters. The first parameter, which is mandatory, is the HTTP method to be used for the request message, `GET` or `POST`. The HTTP method is passed as a literal string, so, it must be quoted. For this application, `GET` will be used. Recall that `GET` is used when there is a relatively small amount of data to be passed to the server. `POST` is used when there are many widgets on the form, making the form data lengthy. The second parameter to `open` is the URL of the response document on the server that will either be or produce the response. In this application, the document will be the response, in the form of plain text. This URL is often just a file name without a path, because the file that produces the response is often in the same directory as the form document. The third parameter specifies whether the request is to be asynchronous or synchronous, with `true` signifying ascynchronous. Because the whole idea of Ajax is to use asynchronous requests, we will always send `true` as the third parameter to `open`, even though `true` is the default value if the parameter is omitted. The last two optional parameters. when used, specify a username and password, which may be needed if a database must be accessed for the response document.

Because the request handler uses the `GET` method and the user-entered zip code must be sent to the server, that zip code must be attached with a question mark to the URL of the response document. Recall that the catenation operator in JavaScript is the plus sign (+). Following is the call to `open` for our application:

```
xhr.open("GET", "getCityState.php?zip=" + zip, true);
```

Notice that a PHP document, `getCityState.php`, will be used to generate the response document.

The final step in the request handler is to actually send the request to the server. This is done with the `send` method of the XHR object, which takes a single parameter. The parameter could be used to send a string or a DOM object to the server to be posted, but that is rarely used. `null` is used as the parameter for our application, as is seen in the following call to send:

```
xhr.send(null);
```

Following is the complete request handler function:

```
// function getPlace
//  parameter: zip code
//  action: create the XMLHttpRequest object, register the
//          handler for onreadystatechange, prepare to send
//          the request (with open), and send the request,
//          along with the zip code, to the server

function getPlace(zip) {
  var xhr = new XMLHttpRequest();
  xhr.onreadystatechange = receivePlace;
  xhr.open("GET", "getCityState.php?zip=" + zip, true);
  xhr.send(null);
}
```

16.2.4 The Response Document

The response document for this application is simple. Rather than using a database that has zip codes, cities, and states, for the sake of simplicity, only a hash with a few entries is used for testing. Recall that the name of the response document used a `.php` extension, so the document must be written in PHP. The actual response is produced with a PHP `print` statement. Any output produced by the response document will be returned to the requester browser. Because the widget form value was sent with `GET`, the predefined PHP array `$_GET` can be used to retrieve it. Following is the complete response document:

```
<?php
// getCityState.php
//  Gets the form value from the "zip" widget, looks up the
//  city and state for that zip code, and prints it for the
//  form

  $cityState = array("81611" => "Aspen, Colorado",
                     "81411" => "Bedrock, Colorado",
                     "80908" => "Black Forest, Colorado",
                     "80301" => "Boulder, Colorado",
                     "81127" => "Chimney Rock, Colorado",
                     "80901" => "Colorado Springs, Colorado",
                     "81223" => "Cotopaxi, Colorado",
                     "80201" => "Denver, Colorado",
                     "81657" => "Vail, Colorado",
                     "80435" => "Keystone, Colorado",
                     "80536" => "Virginia Dale, Colorado",
```

```
                    );
  $zip = $_GET["zip"];
  if (array_key_exists($zip, $cityState))
    print $cityState[$zip];
  else
    print " , ";
?>
```

Notice that the response data is a string consisting of a city name, followed by a comma, a space, and a state name. Also, `getCityState` checks to see if it "knows" the zip code. If it does, it returns the city and state; otherwise it returns blanks, which results in the form elements for city and state remaining blank.

16.2.5 The Receiver Phase

The receiver phase is implemented as a JavaScript function with no parameters. The function's task is to fetch the server response, which in this case is plain text, split it into a city name and a state name, and set the city and state text boxes to the results.The response text is available in the `responseText` property of the XHR object.

The receiver function obviously must be able to access the XHR object, which was created in the request phase function, `getPlace`. If the XHR object is created as a global and both `getPlace` and the receiver function are placed in a file with the declaration of the XHR object, that would provide both with access. Unfortunately, that would allow another problem to occur. It is possible that more than one request could be made before the response occurs, meaning the earlier XHR object could be overwritten by the creation of another one. One solution to this problem is to register the receiver function definition directly; that is, to place the definition of the receiver function in the request function. So, instead of registering the name of a function, whose definition is elsewhere, we do not name the function and we assign its definition directly to `onreadystatechange`. We will do this, but we first develop the receiver function actions. Note that such a nameless function is sometimes called a *closure*. Such a function inherits the environment in which it is defined, which in our case gives it access to the XHR object.

The first action of the receiver function is to determine the value of the `readyState` property of the XHR object. Recall that a value of `4` means the response is completed. Therefore, the receiver function encapsulates all of its actions in the `then` clause of an `if` construct, where the `if` condition is `xhr.readyState == 4`. This function will be called several times when the value of `readyState` is less than `4`. For these calls, the receiver function does nothing. When `readyState` becomes `4`, the receiver function gets the response text, uses the `split` method to separate it into city and state, and sets the text boxes for city and state to the values produced by `split`. The complete nameless receiver function follows:

```
function () {
  if (xhr.readyState == 4) {
    var result = xhr.responseText;
    var place = result.split(', ');
    document.getElementById("city").value = place[0];
    document.getElementById("state").value = place[1];
  }
}
```

The JavaScript file, `popcornA.js`, includes the request function, `getPlace`, with its embedded receiver functions.

```
// popcornA.js
//  Ajax JavaScript code for the popcornA.html document

/*******************************************************/
// function getPlace
//  parameter: zip code
//  action: create the XMLHttpRequest object, register the
//          handler for onreadystatechange, prepare to send
//          the request (with open), and send the request,
//          along with the zip code, to the server
//  includes: the anonymous handler for onreadystatechange,
//            which is the receiver function, which gets the
//            response text, splits it into city and state,
//            and puts them in the document

function getPlace(zip) {
  var xhr = new XMLHttpRequest();

// Register the embedded receiver function as the handler
  xhr.onreadystatechange = function () {
    if (xhr.readyState == 4) {
      var result = xhr.responseText;
      var place = result.split(', ');
      document.getElementById("city").value = place[0];
      document.getElementById("state").value = place[1];
    }
  }
  xhr.open("GET", "getCityState.php?zip=" + zip);
  xhr.send(null);
}
```

Figure 16.3 shows the displayed form after the zip code has been entered. Figure 16.4 shows the displayed form after the zip code text box has lost focus.

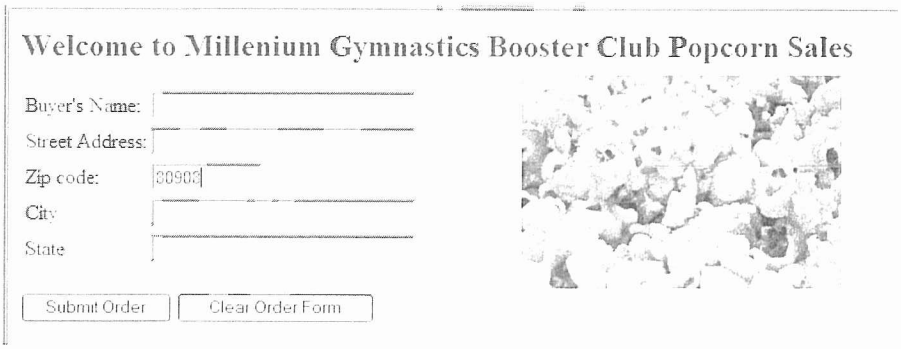


Figure 16.3 Display of the form after the zip code has been entered

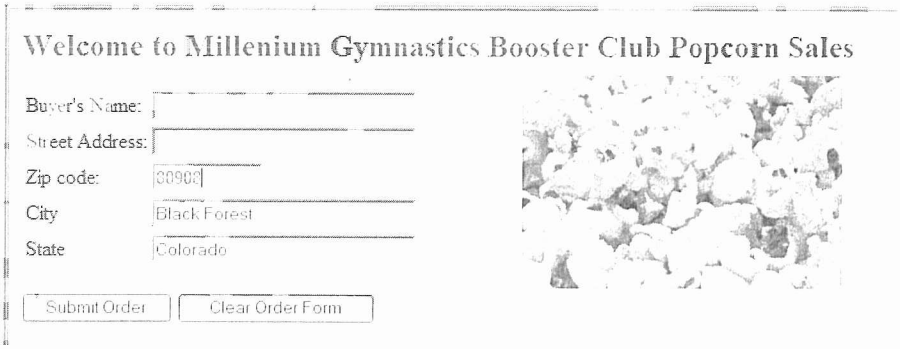


Figure 16.4 Display showing the city and state provided implicitly

16.2.6 Cross-Browser Support

The application demonstrated in Section 16.2.5 works correctly under FX2 and IE7 browsers. However, it does not work with earlier IE browsers. Because there are still a large number of people who use IE6 and to a lesser extent IE5, making Ajax work under those browsers must be considered.

The problem with IE5 and IE6 is that they do not support the XHR object named `XMLHttpRequest`. They do, however, support a similar object with a different name. So, to make Ajax applications operate correctly on both of these earlier browsers and also all contemporary browsers, these differences must be

taken into account. The name of the IE5 and IE6 object is `Microsoft.XMLHTTP`, and it is an `ActiveXObject`.

Actually, `XMLHTTP` is the name of the original object used for asynchronous requests, invented by Microsoft. When Netscape got around to copying this idea, they named their object `XMLHttpRequest`, and other browser makers followed. Finally, in IE7, Microsoft changed to the name used by the others.

The code to create the original object (used in IE5 and IE6) is as follows:

```
xhr = new ActiveXObject("Microsoft.XMLHTTP");
```

The code can determine whether `XMLHttpRequest` is supported by testing `window.XMLHttpRequest`. If not, we assume it is IE5 or IE6 and create the `XMLHTTP` object. The cross-browser version of the `getPlace` function is as follows:

```
// function getPlace
//  parameter: zip code
//  action: create the SMLHttpRequest object, register the
//          handler for onreadystatechange, prepare to send
//          the request (with open), and send the request,
//          along with the zip code, to the server

function getPlace(zip) {

// Get the object for all browsers except IE5 and IE6
  if (window.XMLHttpRequest)
    xhr = new XMLHttpRequest();

// Otherwise get the object for IE5 and IE6
  else
    xhr = new ActiveXObject("Microsoft.XMLHTTP");

// Do the usual xhr stuff
  xhr.onreadystatechange = receivePlace;
  xhr.open("GET", "getCityState.php?zip=" + zip);
  xhr.send(null);
}
```

16.3 Rails with Ajax

In at least one sense, the idea of Rails and Ajax together does not sound very attractive. After all, Rails is all about Ruby, and Ajax uses JavaScript, and both use XHTML and possibly XML. Mixing XHTML and a programming lan-

guage is an idea with some downsides, so how could mixing XHTML and *two* programming languages be good? Well, it's not as bad as it sounds, primarily because Rails includes a JavaScript library of utilities that allows most of the JavaScritpt to stay in the library and out of the application's listings. The library is named Prototype and it can be used in any Rails template document that includes the following directive:

```
<%= javascript_include_tag "prototype" %>
```

There are some problems with using Rails and Ajax to update a displayed document. Not all elements can be modified or replaced on all browsers using only the Rails tools. The content of an element is changed by Rails by changing the `innerHTML` property. But with Microsoft browsers, `innerHTML` is read-only for some elements, in particular most of the elements associated with tables. One element whose content can be changed successfully on all recent browsers is `div`. In fact, `div` is frequently used by Rails/Ajax applications.

Rails implements Ajax in a sequence of processes, explained briefly as follows:

1. The sequence is triggered, either by a user event, such as clicking a button or link or making changes to a form widget, or a timer reaching its specified value.
2. Data associated with the triggering event, often a form element's value or the value of the whole form, is sent asynchronously to a controller method, often called an *action handler*, on the server using an XHR.
3. The controller action handler performs some operation and returns an XML or XHTML document, or part of one, or perhaps just some plain text to the browser.
4. The JavaScript code on the browser receives the XML, XHTML, or plain text and uses it to update part of the currently displayed document, often by changing a CSS property.

Once again we use the abbreviated popcorn sales form, this time to illustrate the use of Rails and Ajax. The objective of the application is the same as in Section 16.2, to help the user fill a form by providing the city and state parts as soon as the zip code is entered. The entry of the zip code triggers the Ajax process of sending an XHR object to the server, along with the entered zip code. The Rails controller on the server uses the zip code to look up the city and state, which it sends back to the browser in its response. Fortunately, most of the code to support these activities is included in the Prototype library.

16.3.1 The Initial Form Document

This application is named `popcornA`. and the controller is named `popcorn`. Rails is used to generate the skeletal application, including the empty controller class, `PopcornController`. The empty method, `show_form`., is added to this controller. This initial version of the controller is as follows:

```
# popcorn_controller.rb
class PopcornController < ApplicationController
  def show_form
  end
end
```

The initial popcorn order form is presented by the template document, `show_form.rhtml`, which is placed in the `popcorn` subdirectory of the `views` directory of the application. This document is as follows:

```
<?xml version = "1.0" encoding = "utf-8" ?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- show_form.rhtml
     A rails/Ajax example form;
     This describes popcorn sales form page which uses
     Ajax and the zip code to fill in the city and state
     of the customer's address
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Popcorn Sales Form (Ajax) </title>
    <style type = "text/css">
      img {position: absolute; left: 400px;  top: 50px;}
    </style>
    <%= javascript_include_tag "prototype" %>
  </head>
  <body>
    <h2> Welcome to Millenium Gymnastics Booster Club Popcorn
         Sales
    </h2>
    <form action = "">

<!-- A borderless table of text widgets for name and address -->
      <table>
        <tr>
          <td> Buyer's Name: </td>
          <td> <input type = "text"  name = "name"
                      size = "30" />
          </td>
        </tr>
        <tr>
          <td> Street Address: </td>
```

```
          <td> <input type = "text"  name = "street"
                      size = "30" />
          </td>
        </tr>
        <tr>
          <td> Zip code: </td>
          <td> <input type = "text"  name = "zip"
                      id = "zip"  size = "10" />
          </td>
        </tr>
        <tr>
          <td> City </td>
          <td> <input type = "text"  name = "city"
                      id = "city"  size = "30" />
          </td>
        </tr>
        <tr>
          <td> State </td>
          <td> <input type = "text"  name = "state"
                      id = "state"  size = "30" />
          </td>
        </tr>
      </table>
      <p />

      <img src = "popcorn.jpg"  alt = "picture of popcorn" />

<!-- The submit and reset buttons -->
      <p>
        <input type = "submit"  value = "Submit Order" />
        <input type = "reset"  value = "Clear Order Form" />
      </p>
    </form>
  </body>
</html>
```

Notice that this document uses the `javascript_include_tag` to include the Prototype library.

16.3.2 Triggering Ajax

Rails includes several helper methods to trigger Ajax processing. Two of the most commonly used of these are `link_to_remote`, which triggers when a specific link is taken, and `observe_field`, which triggers when a specific form element is changed. The Rails helper functions for Ajax are JavaScript functions

that are wrapped in Ruby methods. This explains the form of their parameters, which are often Ruby symbols.

Only the `observe_field` triggering method is described in this chapter. The `observe_field` method takes the id of the widget to be watched as its first parameter. This parameter is given as either a literal string or a symbol. For example, if `"zip"` is the id attribute value for a widget to watch, the following could be used:

```
observe_field("zip", ...)
```

The other parameters to `observe_field`, most of which are optional, appear in the form of a list of elements of a hash literal. The keys to the hash are given as symbols. The forms of the values vary among the parameters. Brief descriptions of the most commonly used parameters are as follows:

`:url`—Specifies as its value the URL of where the result from the server is to be posted. Its value is given in the form of a hash literal (a list of hash elements delimited with braces). In many cases, the value of `:url` is a hash literal with a single element consisting of the key `:action` and the value being the name of the action method of the controller that will handle the returned value from the Ajax request. For example:

```
:url => {:action => process_form}
```

`:update`—Specifies the id of the element whose value is to be changed by the XHR response value. Note that only one element can be changed. When the `:update` parameter is specified, the controller action specified in the `:url` parameter produces the response with the `render` method. If what is to be rendered is text, the parameter to `render` is a the hash literal element with the symbol key `:text` followed by `=>` and whatever text is to be rendered. For example, the following example returns a string of text that includes two literal strings and a date/time value:

```
render :text => "<p>The time is" + DateTime.now.to_s +
                "</p>"
```

The string parameter to `render` is assigned to the `responseText` property of the XHR object.

`:with`—A JavaScript expression specifying the first parameter for the XHR request object. If given as a literal string, say `"zip"`, it takes the form of `"'zip'=value"`, specifying the form key to which the value should be assigned.

`:frequency`—The frequency in seconds at which changes to the specified widget will be detected. Not setting this parameter at all or setting it to a value equal to or less than zero will use an event (`changed` for text boxes and `click` for buttons), rather than time-based observations.

`:complete`, `:success`, `:failure`—These are used to specify a callback function to be called when the Ajax operation is completed. The choice among the three depends on how termination occurred. If the callback function should be called regardless of how the operation finished, the `:complete` key is used. If the function is to be called only if the operation was completed and successful, the `:success`), key is used. If the function is to be called only if the operation was completed but failed, the `:failure` key is used. The callback function is a JavaScript function and the call takes the parameter, `request`, which specifies that the XHR object is to be passed to the function. Note that the `:complete`, `:success`, and `:failure` parameters are used as an alternative to `:update`. These dictate that the update is to be done by the specified JavaScript function.

Following is an example of a call to `observe_field`:

```
<%= observe_field("my_button",
                  :update => "name"
                  :url => {:action => process_form}) %>
```

The controller action method for our application, which is called when the XHR response appears, is named `fill_city_state`. The actual insertion of the city and state into the form is complicated by the fact that a controller can only produce a single value and both a city and a state are needed. One simple way around this is to have the controller action render the city and state in a string and to embed a JavaScript function in the template to process the single string of the city and state provided by the controller method. Such a function can be inserted into a document with a `javascript_tag`, which specifies a list of JavaScript code. The end of the code sequence is specified with the `END` flag, which must be specified following the `javascript_tag` with `<<-`. The function can appear in the template in the following form:

```
<%= javascript_tag <<-END
   function update_city_state(xhr) {
     result = xhr.responseText.split(', ');
     document.getElementById('city').value = result[0];
     document.getElementById('state').value = result[1];
   }
 END
%>
```

The JavaScript function is embedded in this way because this document is an `rhtml` document, meaning it will be processed by the Ruby interpreter, `ERb`. If the JavaScript had been embedded as if the document were an XHTML document, `ERb` would not know how to handle it.

The Prototype library defines an abbreviation for `document.getElementById` method, `$`. So, `$('city')` is the same as `docu-`

ment.getElementById('city'). This is simply an abbreviation, meant to reduce typing and the size of JavaScript code to be transmitted over the Internet. It was not used in update_city_state to avoid confusing the reader. It is mentioned here because most Rails/Ajax applications use it.

Note that in the non-Rails example in Section 16.2, updatting more than one element is not a problem, because the updating is already done with JavaScript.

In the current example, instead of using :update to indicate which element is to be changed, the name of the JavaScript function that will update the form is given in the :complete parameter. Recall that request, which appears as the actual parameter to the JavaScript function update_city_state, specifies that Rails must send the XHR object to the function. The call to the function occurs when the XHR request is complete. The complete call to observe_field is as follows:

```
<%= observe_field("zip",
                  :url => {:action => :fill_city_state},
                  :complete => 'update_city_state(request);',
                  :with => 'zip'
                 ) %>
```

16.3.3 The Controller

The controller action method, fill_city_state, generates the city and the state for the form, given the zip code as a POST parameter. Recall that such a parameter is available to the controller through the params hash. If the string 'zip' is provided as the value or the :with parameter of observe_field, the controller can fetch it with params[:zip]. Then fill_city_state can look up the city and state and provide it for the template with the render_text library function. For our example, fill_city_state includes a hash of some cities and states and zip codes, just for testing and illustration purposes. The complete controller is as follows:

```
# popcorn_controller.rb
#  The controller for the popcornA application
#  When the user enters a zip code in the popcorn
#  order form, this application puts in the city
#  and state for the user

class PopcornController < ApplicationController
  @@city_state_data = {"81611" => "Aspen, Colorado",
                       "81411" => "Bedrock, Colorado",
                       "80908" => "Black Forest, Colorado",
                       "80301" => "Boulder, Colorado",
```

```
                        "81127" => "Chimney Rock, Colorado",
                        "80901" => "Colorado Springs, Colorado",
                        "81223" => "Cotopaxi, Colorado",
                        "80201" => "Denver, Colorado",
                        "81657" => "Vail, Colorado",
                        "80435" => "Keystone, Colorado",
                        "80536" => "Virginia Dale, Colorado"
                      }
  def show_form
  end

# The fill_city_state method
#  Renders the city and state

  def fill_city_state
    zip = params[:zip]

# If possible, set city_state to the city and state
    if @@city_state_data.has_key?(zip)
      city_state = @@city_state_data[zip]

# Otherwise, set it to blanks
    else
      city_state = " , "
    end

# Render it
    render :text => city_state
  end # end of the fill_city_state method
end
```

In general, as in any Rails application, the new partial document can be produced by either the controller method or in a template document, with the template document being the preferred approach most of the time. If the template document does it, the action method has the following single statement:

```
render(:layout => false)
```

The parameter is used to indicate that only a part of the displayed document is to be updated, so there is no need for any Rails layout wrappers.

If the action method is to produce the document fragment, it does it with `render`.

An alternative solution to the problem of wanting to update two elements in the form and `:update` being restricted to a single element is to replace the whole form, which would require a second `rhtml` document for the updated form. Of course, that would be silly for our example, because the form is nearly

the whole initial document, so replacing it would only be marginally better than requesting a whole new document without Ajax.

Summary

Ajax is a relatively new technology for building Web applications that can make use of rapid updates to parts of documents. Asynchronous requests are made to the server, allowing users to continue to interact with the browser while the request is being handled. JavaScript code is used to create the Ajax request object. Any server software can be used to generate the response. JavaScript is again used to receive the new partial document and insert it into the currently displayed document.

Internet Explorer browsers prior to IE7 must be handled differently, because the object used to make asynchronous requests has a different name in those browsers.

Rails can be used to build Web applications that use Ajax. Rails includes a JavaScript library of helper functions to make this process easier. Two of the functions that can be used to trigger asynchronous requests are `link_to_remote` and `observe_field`. The `observe_field` function specifies the field to be watched, the action method of the controller to handle the request, and a timer for the watch process, among other things. The controller method whose name is given in the call to `observe_field` provides the response to the request. A JavaScript function can be embedded in the displayed document to insert the new parts in that document.

Review Questions

16.1 What is the goal of the use of Ajax in a Web application?

16.2 What does it mean for a request to be asynchronous?

16.3 What new languages are used to program Web applications that use Ajax?

16.4 What new software must be installed on a browser or server to run Web applications that use Ajax?

16.5 What is required for an Ajax application to run on both IE6 and the latest versions of browsers?

16.6 What is stored in the `readyState` property of an XHR object?

16.7 Under what circumstances would one use the `POST` method for an Ajax request?

16.8 How are parameters passed in a `GET` Ajax request?

16.9 For what purpose is the `javascript_include_tag` used in a Rails application that uses Ajax?

16.10 With what two functions can an Ajax request be triggered?

16.11 What is specified with the `:url` parameter to `observe_field`?

16.12 What is specified with the `:update` parameter to `observe_field`?

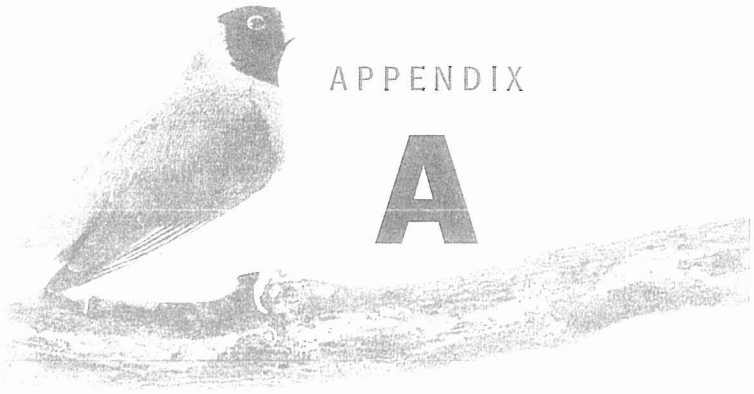
Exercises

16.1 Explain the two characteristics of Ajax that help it achieve its goals.

16.2 Modify the example application of Section 16.2 to have it provide the addresses of repeat customers, using a hash of names and addresses.

16.3 Modify the example application of Section 16.3 to have it provide the addresses of repeat customers, using a hash of names and addresses.

16.4 Build a Rails application that uses Ajax to provide a personalized greeting to a user when the user types in his or her name.



APPENDIX

A

Introduction to Java

A.1 Overview of Java
A.2 Data Types and Structures
A.3 Classes, Objects, and Methods
A.4 Interfaces
A.5 Exception Handling
Summary

This appendix provides a quick introduction to Java for programmers who are familiar with C++ and object-oriented programming. It covers only a small part of Java, focusing on the features needed to understand Java programs similar to those discussed in this book. In some cases—for example, concurrency—the discussion of a topic can be found in the chapter of the book in which it is used, rather than in this appendix.

This appendix begins with a broad overview of the features and capabilities of Java. The data types and data structures of Java are then discussed, as well as the control statements. Next, it introduces the class definitions of Java, including some of the details of data and method definitions. Java interfaces, which provide a limited kind of multiple inheritance, are then discussed. This is followed by a description of Java exception handling.

A.1 Overview of Java

Java is based on C++, so it is closely related to that language. However, some parts of C++ were left out of the design of Java in an attempt to make it smaller and simpler. Other C++ features were redesigned in Java. Java also includes some constructs that are not part of C++. In comparison with C++, Java can be characterized by the following categories of differences: exclusive support for object-oriented programming, no user-defined overloading, implicit deallocation of heap objects, use of interfaces, lack of pointers, and far fewer type coercions.

C++ was designed originally as an extension to C to provide support for object-oriented programming. Because virtually nothing was left out of C, C++ supports procedure-oriented programming as well as object-oriented programming. Java does not support procedure-oriented programming. In practical terms, this means that subprograms in Java can only appear as methods defined in class definitions. The same is true for data definitions. Therefore, all data and functionality are associated with classes, and therefore with objects.

C++ allows users to define new operations that are specified by existing operator symbols. For example, if a user defines a class to support complex numbers, he or she can overload the definitions of + and – so that they can be used as binary operators for complex objects. For the sake of simplicity, Java does not allow user-defined operator overloading.

In C++, user programs can both allocate and deallocate storage from the heap. This leads to a number of different programming problems, including the possibility of dangling pointers. A dangling pointer is one that is pointing to a memory cell that has been explicitly deallocated from its previous use and possibly reallocated to a new use. Some of these problems are avoided by making heap storage deallocation a system responsibility rather than a user one. In Java, all heap storage deallocation is implicit and a technique named *garbage collection* is used to reclaim heap storage that has been implicitly deallocated.

In C++, a user program can define a class to extend two or more different classes, thereby making use of multiple inheritance. Although multiple inheritance is sometimes convenient, it has some disadvantages, among them the possibility of designing programs whose complexity makes them difficult to understand. For this reason, Java does not support multiple inheritance. In its place, Java has interfaces, which provide some of the functionality of multiple inheritance. Interfaces are discussed in Section A.4.

Pointers are notoriously risky, especially when pointer arithmetic is allowed. Java does not include pointers. Instead, Java provides references, which are also supported by C++, though in a somewhat different way. Reference variables in Java are used to reference objects, rather than memory cells, so they cannot be used as the operands of arithmetic operators. This, in conjunction with the lack of a deallocation operator for heap objects, makes references far safer than the pointers of C++.

In C++, as in many other programming languages, it is legal to assign a value of any numeric type to a variable of any other numeric type. This requires

the compiler to build type conversion code, called *coercions*, into the program. Half of these conversions are narrowing conversions, in which it may not be possible to convert the value into even an approximation in the new type. For example, in C++ it is legal to assign a `float` value to an `int` variable, although this is a narrowing conversion. For example, `float` values such as `1.23E15` cannot be converted to anything close to that value as an `int` value. Java does not allow narrowing coercions in assignment statements. It is syntactically illegal to write such an assignment statement. This results in an increase in the overall safety of programs written in Java over those written in C++.

The control statements of Java are almost exactly like those in C++. One difference is that control expressions in control statements in Java must have Boolean values, whereas in C++ the control expression can be either Boolean or a numeric type. For example, in Java, the following statement is illegal:

```
if (2 * count) ...
```

Output to the screen from a Java application is through the object `System.out`, which represents the console window associated with the application. This object has two methods, `print` and `println`, which do something similar to what you would expect given their names. Both take a string parameter, but also permit variables as parameters. The values of non-`String` variables that appear in the parameter to `System.out.print` or `System.out.println` are implicitly converted to strings. The `print` method produces a string of output to the screen without attaching a newline character to the end. The `println` method does what `print` does, except that it attaches a newline character to the end. The string parameter to `print` and `println` is often specified as a catenation of several strings, using the + catenation operator. The following method calls illustrate the use of `print` and `println`:

```
System.out.println("Apples are good for you");
System.out.println("You should eat " + numApples +
                   " apples each week");
System.out.print("Grapes ");
System.out.println("are good, too");
```

If `numApples` is 7, these statements produce the following display:

```
Apples are good for you
You should eat 7 apples each week
Grapes are good, too
```

Naming conventions used in Java are as follows:

- Class and interface names begin with uppercase letters.
- Variable and method names begin with lowercase letters.
- Package names are all lowercase letters.
- Constant names are all uppercase letters, with underscores used as separators.

- Except for package and constant names, when a name consists of more than one word, the first letters of all embedded words are capitalized.
- Except for constant names, all but the first letters of embedded words are lowercase.

Java does not have an address-of operator (`&` in C++), a dereference operator (unary `*` in C++), or an operator to return the size of a type or object (`sizeof` in C++).

A.2 Data Types and Structures

In both C++ and Java, there are two kinds of data values: primitives and objects. This is a compromise design, for it provides efficiency in arithmetic operations on primitive values at the expense of complicating the object model of the language. Arithmetic operations can be done very quickly on primitive values, but are more costly when the operands are objects.

C++ has three different kinds of variables for objects: those whose value is a stack-allocated object, pointers that reference heap-allocated objects, and references that reference heap-allocated objects. In Java, there is only one way to reference an object, namely, through a reference variable. This simplicity is possible because all objects are allocated from the heap and there are no pointer variables in Java.

The Java primitive types are `int`, `float`, `double`, `char`, and `boolean`. Operations on primitive values are similar to those in other programming languages. Each of the primitive types has a corresponding *wrapper class*, which is used when it is convenient to treat a primitive value as an object.[1] The Java wrapper classes are named with the name of the associated primitive type, except that the first letter is capitalized. For example, the wrapper class for `double` is `Double`. An object of a wrapper class is created with the `new` operator and the class's constructor, as shown in the following example:

```
Integer wrapsum = new Integer(sum);
```

One of the purposes of wrapper classes is to provide methods that operate on primitive values. For example, a `float` value can be converted to a string by creating an object for it and using the `toString` method on that object. To convert the `float` value `speed` to a `String` object, the following could be used:

```
float speedObj = new Float(speed);
String speedStr = speedObj.toString();
```

As stated previously, all objects are referenced through reference variables. Reference variables are defined the same way as primitive variables. For example:

```
int sum;
String str1;
```

1. These classes are called wrapper classes because in effect they wrap a primitive value so it looks like an object.

In this example, `sum` is a primitive variable of type `int`, and `str1` is a reference variable that can reference a `String` object, initially set to `null`.

Although an array of characters can be created and used in Java, it is more convenient to use the `String` and `StringBuffer` classes for character strings. `String` objects are immutable strings of characters. They can be created in two ways: either with the `new` operator or implicitly, as illustrated with the following declarations:

```
String greet1 = new String("Guten Morgan");
String greet2 = "Guten Morgan";
```

These two strings are equivalent. All Java `String` and `StringBuffer` objects use 2 bytes per character because they use the Unicode character codings, which are 16 bits wide.

String catenation, which is specified with the plus operator (+), can be used on `String` objects, as shown in the following example:

```
greet3 = greet3 + " New Year";
```

There are a number of methods that can be called through `String` objects to perform more or less standard string operations—for example, `charAt`, `substring`, `concat`, and `indexOf`. The `equals` method of `String` must be used to compare two strings for equality. Because strings are objects, the `==` operator is of no use between strings.

If a string must be manipulated, it cannot be a `String` object (because `String` objects cannot be changed). For this situation, a `StringBuffer` object can be used. `StringBuffer` objects are created with `new`, as shown in the following example:

```
StringBuffer greet3 = new StringBuffer("Happy");
```

The `StringBuffer` class has a collection of methods to manipulate its objects. Among them are `append`, which appends a given value to the end of the object; `delete`, which deletes one or more characters from the object; and `insert`, which inserts a value into its string object. In the cases of `append` and `insert`, if the given parameter is not a string, it is implicitly converted to a string.

In Java, arrays are objects of a class that has some special functionality. Array objects, like all other objects, are always referenced through reference variables and are always allocated on the heap. Array objects can be created with statements having the following form:

```
element_type array_name[] = new element_type[length];
```

For example:

```
int[] list1 = new int[100];
float[] list2 = new float[10];
```

If an array reference variable has been previously created, as with

```
int[] list3;
```

an object can be created with

```
list3 = new int[200];
```

As with other related languages, the subscript ranges of Java arrays always begin with zero. In a departure from C++, all references to array elements are checked to be sure the subscript values are within the defined subscript ranges of the array. Therefore, it is not possible to reference or assign an array element that does not exist. When a subscript that is out of range is detected, the exception `ArrayIndexOutOfBoundsException` is thrown. Java exception handling is discussed in Section A.5.

Java does not have the `struct` and `union` data structures that are part of C++. It also does not have the `unsigned` types or the `typedef` declaration.

A.3 Classes, Objects, and Methods

There are several important differences between C++ class definitions and those of Java. All Java classes have a parent class, whereas in C++ a class does not need to have a parent. The parent of a class is specified in the class definition with the `extends` reserved word. The general form of a class definition is

[*modifiers*] `class` *class_name* [`extends` *parent_class*] `{ ... }`

The square brackets here indicate that what they delimit is optional. Three different modifiers can appear at the beginning of a class definition: `public`, `abstract`, and `final`. The `public` modifier makes the class visible to classes that are not in the same package (packages are described later in this section). The `abstract` modifier specifies that the class cannot be instantiated. An abstract class is designed to be a class model that can be extended by nonabstract classes. The `final` modifier specifies that the class cannot be extended.

The root class of all Java classes is `Object`. A class definition that does not specify a parent is made a subclass of `Object`.

In C++, the visibility of variables and member functions (methods) defined in classes is specified by placing their declarations in `public`, `private`, or `protected` clauses. In Java, these same reserved words are used, but on individual declarations rather than on clauses. The meanings of these access modifiers are the same as in C++.

In addition to the access modifiers, a variable declaration can include the `final` modifier, which specifies that the variable is actually a constant, in which case it must be initialized. Java does not use C++'s `const` reserved word to specify constants.

In Java, all methods are defined in a class. Java class methods are specified by including the `static` modifier in their definitions. Any method without `static` is an instance method. Methods can also have several other modifiers. Among these are `abstract` and `final`. The `abstract` modifier specifies that the method is not defined in the class. The `final` modifier specifies that the method cannot be overridden.

Whereas C++ depends on classes as its only encapsulation construct, Java includes a second one at a level above classes, the *package*. Packages can contain more than one class definition, and the classes in a package are similar to the friend classes of C++. The entities defined in a class that are public or protected or have no access specifier are visible to all other classes in the package. This is an expansion of the definition of protected as used in C++, in which protected members are visible only in the class in which they are defined and in subclasses of that class. Entities without access modifiers are said to have *package scope*, because they are visible throughout the package. Therefore, Java has less need for explicit friend declarations and in fact does not include either the friend functions or friend classes of C++. Packages, which often contain libraries, can be defined in hierarchies. The standard class libraries of Java are defined in a hierarchy of packages.

A file whose class definitions are to be put in a named package includes a package declaration, as shown in the following example:

```
package cars;
```

The external visibility of entities in a class is controlled by the accessibility modifiers on the entities. Entities from other classes that are visible can be referenced through their complete name, which begins with the name of the package in which the class is defined and includes the name of the class in which the entity is defined. For example, if we have a package named `weatherpkg`, which includes a class named `WeatherData`, which defines a public variable named `avgTemp`, `avgTemp` can be referenced in any other class where it is visible with the following:

```
weatherpkg.WeatherData.avgTemp
```

An `import` statement provides a way to abbreviate such imported names. For example, suppose we include the following statement in our program:

```
import weatherpkg.WeatherData;
```

Now the variable `avgTemp` can be accessed directly (with just its name). The `import` statement can include an asterisk instead of a class name, in which case all classes in the package are imported. For example:

```
import weatherpkg.*;
```

A Java application program is a compiled class that includes a method named `main`. The `main` method of a Java application is where the Java interpreter begins. The following illustrates the simplest kind of Java application program:

```
public class Trivial {
  public static void main (String[] args) {
    System.out.println("A maximally trivial Java
                        application");
  }
}
```

The modifiers on the `main` method are always the same. It must have public accessibility, and it cannot be extended. The `void` modifier indicates that `main` does not return a value. The only parameter to `main` is an array of strings that contains any command-line parameters from the user. In many cases, command-line parameters are not used. When they are used, the interpreter passes them to `main` as strings.

In C++, methods can be defined in a somewhat indirect way: The protocol is given in the class definition, but the definition of the method appears elsewhere. In Java, however, method definitions must appear in their associated classes.

As with C++, Java constructors have the same names as the classes in which they appear. C++ uses destructor methods to deallocate heap storage for instance data members, among other things. Because Java uses implicit heap deallocation, it does not have destructors.

In some object-oriented programming languages, including C++, method calls can be bound to methods either statically (at compile time) or dynamically (during runtime). In C++, the default binding of method calls to methods is static. Only methods defined to be virtual are dynamically bound. In Java, the default is dynamic.

Objects of user-defined classes are created with `new`. As with array objects, a reference variable is required to access an object, but both the reference variable and the object can be created in the same statement. For example:

```
MyClass myObject1;
myObject1 = new MyClass();
MyClass myObject2 = new MyClass();
```

The two reference variables, `myObject1` and `myObject2`, refer to new objects of class `MyClass`.

As is the case with C++, Java classes can have instance or class variables or both. There is a single version of a class variable per class; there is an instance variable for every instance of the class in which it is defined. Both instance and class variables that are not explicitly initialized in their declarations are implicitly initialized. Numeric variables are implicitly initialized to zero, Boolean variables are initialized to `false`, and reference variables are initialized to `null`.

Inside the methods of a class, instance variables are referenced directly. In other classes, instance variables are referenced through the reference variables that point at their associated objects. For example:

```
class MyClass extends Object {
   public int sum;
   ...
}
MyClass myObject = new MyClass();
```

In other classes that either import `MyClass` or are defined in the same package, the instance variable `sum` can be referenced as follows:

```
myObject.sum
```

Similar to class methods, class variables are specified by preceding their declarations with the `static` reserved word.

The following is an example of a class definition that illustrates some of the aspects of Java we have discussed. It implements a stack in an array.

```
import java.io.*;
class Stack_class {
  private int [] stack_ref;
  private int max_len,
              top_index;
  public Stack_class() {  // A constructor
    stack_ref = new int [100];
    max_len = 99;
    top_index = -1;
  }
  public void push(int number) {
    if (top_index == max_len)
      System.out.println("Error in push--stack is full");
    else stack_ref[++top_index] = number;
  }
  public void pop() {
    if (top_index == -1)
      System.out.println("Error in pop--stack is empty");
    else --top_index;
  }
  public int top() {return (stack_ref[top_index]);}
  public boolean empty() {return (top_index == -1);}
}
```

An example class that uses `Stack_class` follows:

```
public class Tst_Stack {
  public static void main(String[] args) {
    Stack_class myStack = new Stack_class();
    myStack.push(42);
    myStack.push(29);
    System.out.println("29 is: " + myStack.top());
    myStack.pop();
    System.out.println("42 is: " + myStack.top());
    myStack.pop();
    myStack.pop();  // Produces an error message
  }
}
```

We must note here that a stack is a silly example for Java because the Java library includes a class definition for stacks.

A.4 Interfaces

Java directly supports only single inheritance. However, it includes a construct similar to a virtual class, called an *interface*, that provides something closely related to multiple inheritance. An interface definition is similar to a class definition except that it can contain only named constants and method declarations (not definitions). So, an interface is no more than what its name indicates, just the specification of a class. (Recall that a C++ abstract class can have instance variables, and all but one of the methods can be completely defined.) The typical use of an interface is to define a class that inherits some of the methods and variables from its parent class and implements an interface as well.

Applets are programs that are interpreted by a Web browser after being downloaded from a Web server. Calls to applets are embedded in the HTML code that describes an HTML document. These applets all need certain capabilities, which they can inherit from the predefined class `Applet`. When an applet is used to implement animation, it is often defined to run in its own thread of control. This concurrency is supported by a predefined class named `Thread`. However, an applet class being designed to use concurrency cannot inherit from both `Applet` and `Thread`. Therefore, Java includes a predefined interface named `Runnable` that supplies the interface (but not the implementation) to some of the methods of `Thread`. The syntax of the header of such an applet is exemplified by the following:

```
public class Clock extends Applet implements Runnable
```

Although this code appears to provide multiple inheritance, in this case it requires a further complication. For an object of the `Clock` class to run concurrently, a `Thread` object must be created and connected to the `Clock` object. The messages that control the concurrent execution of the `Clock` object must be sent to the corresponding `Thread` object. This is surely an inelegant and potentially confusing necessity.

A.5 Exception Handling

Java's exception handling is based on that of C++, but is designed to be more faithful to the object-oriented language paradigm.

A.5.1 Classes of Exceptions

All Java exceptions are objects of classes that are descendants of the `Throwable` class. The Java system includes two system-defined exception classes that are subclasses of `Throwable`: `Error` and `Exception`. The `Error` class and its descendants are related to errors that are thrown by the Java interpreter, such as running out of heap memory. These exceptions are never thrown by user programs, and they should never be handled there. The two system-defined direct descendants of `Exception` are `RuntimeException` and

`IOException`. As its name indicates, `IOException` is thrown when an error has occurred in an input or output operation, all of which are defined as methods in the various classes defined in the package `java.io`.

System-defined classes that are descendants of `RuntimeException` exist. In most cases, `RuntimeException` is thrown when a user program causes an error. For example, `ArrayIndexOutOfBoundsException`, which is defined in `java.util`, is a commonly thrown exception that descends from `RuntimeException`. Another commonly thrown exception that descends from `RuntimeException` is `NullPointerException`.

User programs can define their own exception classes. The convention in Java is that user-defined exceptions are subclasses of `Exception`.

A.5.2 Exception Handlers

The exception handlers of Java have a form similar to those of C++, except that the parameter of every `catch` must be present and its class must be a descendant of the predefined class `Throwable`.

The syntax of the `try` construct in Java is exactly like that of C++.

A.5.3 Binding Exceptions to Handlers

Throwing an exception is quite simple. An instance of the exception class is given as the operand of the `throw` statement. For example, suppose we define an exception named `MyException` as follows:

```
class MyException extends Exception {
  public MyException() {}
  public MyException(String message) {
    super (message);
  }
}
```

The first constructor in this class does nothing. The second sends its parameter to the parent class (specified with `super`) constructor. This exception can be thrown with

```
throw new MyException();
```

The creation of the instance of the exception for the `throw` could be done separately from the `throw` statement, as shown in the following example:

```
MyException myExceptionObject = new MyException();
...
throw myExceptionObject;
```

Using the constructor with the parameter, our new exception could be thrown with

```
throw new MyException
    ("a message to specify the location of the error");
```

The binding of exceptions to handlers in Java is less complex than in C++. If an exception is thrown in the compound statement of a `try` construct, it is bound to the first handler (`catch` function) immediately following the `try` clause whose parameter is the same class as the thrown object or is an ancestor of it. If a matching handler is found, the `throw` is bound to it and is executed.

Exceptions can be handled and then rethrown by including a `throw` statement without an operand at the end of the handler. The newly thrown exception will not be handled in the same `try` where it was originally thrown, so looping is not a concern. This rethrowing is usually done when some local action is useful but further handling by an enclosing `try` clause or a caller is necessary. A `throw` statement in a handler could also throw some exception other than the one that transferred control to this handler; one particular exception could cause another to be thrown.

A.5.4 Exception Propagation

When a handler is found in the sequence of handlers in a `try` construct, that handler is executed and program execution continues with the statement following the `try` construct. If none is found, the handlers of enclosing `try` constructs are searched, innermost first. If no handler is found in this process, the exception is propagated to the caller of the method. If the method call was in a `try` clause, the search for a handler continues in the attached collection of handlers in the clause. Propagation continues until the original caller is found, which in the case of an application program is `main`. If no matching handler is found anywhere, the program is terminated. In many cases, exception handlers include a `return` statement to terminate the method in which the exception occurred.

To ensure that exceptions that can be thrown in a `try` clause are always handled in a method, a special handler can be written that matches all exceptions that are derived from `Exception`, simply by defining the handler with an `Exception` type parameter, as shown in the following example:

```
catch (Exception genericObject) {
...
}
```

Because a class name always matches itself or any ancestor class, any class derived from `Exception` matches `Exception`. Of course, such an exception handler should always be placed at the end of the list of handlers, because it will block the use of any handler that follows it in the `try` construct in which it appears. The search for a matching handler is sequential, and the search ends when a match is found.

The object parameter to an exception handler is not entirely useless, as it may have appeared to be so far in this discussion. During program execution, the Java runtime system stores the class name of every object in the program. The method `getClass` can be used to get an object that stores the class name, which itself can be gotten with the `getName` method. So, we can retrieve the

name of the class of the actual parameter from the `throw` statement that caused the handler's execution. For the handler above, this is done with

```
genericObject.getClass().getName()
```

The message associated with the parameter object, which is created by the constructor, can be obtained with

```
genericObject.getMessage()
```

A.5.5 The throws Clause

The `throws` clause of Java has an appearance and placement (in a program) similar to that of the `throw` specification of C++. However, the semantics of `throws` is completely different from that of the C++ `throw` clause.

The appearance of an exception class name in the `throws` clause of a Java method specifies that that exception class or any of its descendant exception classes can be thrown by the method. For example, when a method specifies that it can throw `IOException`, it means it can throw an `IOException` object or an object of any of its descendant classes, such as `EOFException`.

Exceptions of class `Error` and `RuntimeException` and their descendants are called *unchecked exceptions*. All other exceptions are called *checked exceptions*. Unchecked exceptions are never a concern of the compiler. However, the compiler ensures that all checked exceptions a method can throw are either listed in its `throws` clause or handled in the method. The reason that exceptions of the classes `Error` and `RuntimeException` and their descendants are unchecked is that any method can throw them.

A method cannot declare more exceptions in its `throws` clause than the method it overrides, though it may declare fewer. So, if a method has no `throws` clause, neither can any method that overrides it. A method can throw any exception listed in its `throws` clause, along with any of the exceptions' descendant classes. A method that does not directly throw a particular exception but calls another method that could throw that exception must list the exception in its `throws` clause. This is the reason the `buildDist` method (in the example in Section A.5.6), which uses the `readLine` method, must specify `IOException` in the `throws` clause of its header.

A method that calls a method that lists a particular checked exception in its `throws` clause has three alternatives for dealing with that exception. First, it can catch the exception and handle it. Second, it can catch the exception and throw an exception that is listed in its own `throws` clause. Third, it can declare the exception in its own `throws` clause and not handle it, which effectively propagates the exception to an enclosing `try` clause, if there is one, or to the method's caller if there is no enclosing `try` clause.

Java has no default exception handlers, and it is not possible to disable exceptions.

A.5.6 An Example

The following example program illustrates two simple uses of exception handlers. The program computes and prints a distribution of input grades by using an array of counters. There are ten categories of grades (0–9, 10–19, . . . , 90–100). The grades themselves are used to compute indexes into an array of counters, one for each grade category. Invalid input grades are detected by trapping indexing errors in the counter array. A grade of 100 is special in the computation of the grade distribution, because the categories all have ten possible grade values, except the highest, which has eleven (90, 91, . . . , 100). (The fact that there are more possible A grades than Bs or Cs is conclusive evidence of the generosity of teachers.) The grade of 100 is also handled in the same exception handler that is used for invalid input data. Following is a Java class that implements this algorithm:

```
import java.io.*;
// The exception definition to deal with the end of data
class NegativeInputException extends Exception {
  public NegativeInputException() {
    System.out.println("End of input data reached");
  }  //** end of constructor
}  //** end of NegativeInputException class
class GradeDist {
  int newGrade,
      index,
      limit_1,
      limit_2;
  int [] freq = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
void buildDist() throws IOException {
//  Input: A list of integer values that represent
//         grades, followed by a negative number
// Output: A distribution of grades, as a percentage for
//         each of the categories 0-9, 10-19, ...,
//         10-100.
  DataInputStream in = new DataInputStream(System.in);
  try {
    while (true) {
      System.out.println("Please input a grade");
      newGrade = Integer.parseInt(in.readLine());
      if (newGrade < 0)
        throw new NegativeInputException();
      index = newGrade / 10;
      try {
        freq[index]++;
      }  //** end of inner try clause
```

```
      catch(ArrayIndexOutOfBoundsException) {
        if (newGrade == 100)
          freq [9]++;
        else
          System.out.println("Error - new grade: " +
                             newGrade + " is out of range");
      }  //** end of catch (ArrayIndex...
    }  //** end of while (true) ...
  }  //** end of outer try clause
  catch(NegativeInputException) {
    System.out.println ("\nLimits    Frequency\n");
    for (index = 0; index < 10; index++) {
      limit_1 = 10 * index;
      limit_2 = limit_1 + 9;
      if (index ==9)
        limit_2 = 100;
      System.out.println("" + limit_1 + " - " +
        limit_2 + "       " + freq [index]);
    }  //** end of for (index = 0; ...
  }  //** end of catch (NegativeInputException ...
}  //** end of method buildDist
```

The exception for a negative input, `NegativeInputException`, is defined in the program. Its constructor displays a message when an object of the class is created. Its handler produces the output of the method. The `ArrayIndexOutOfBoundsException` is predefined and is thrown by the interpreter. In both cases, the handler does not include an object name in its parameter. In neither case would a name serve any purpose. Note that all handlers get objects as parameters, but they are often not useful.

Summary

Although Java is based on C++, it differs from that language in a variety of ways. The primary differences are Java's exclusive support for object-oriented programming, its lack of user-defined overloaded operators, its implicit deallocation and reclamation of heap objects, its interfaces, its lack of pointers, and its lower number of type coercions in assignment statements. Most of these differences were motivated by the perceived safety risks of C++.

Like C++, Java has primitive types and objects. Character strings can be stored as either `String` or `StringBuffer` objects, where `String` objects cannot be changed but `StringBuffer` objects can. Arrays are objects with special behavior. Array indices are always checked for range in Java.

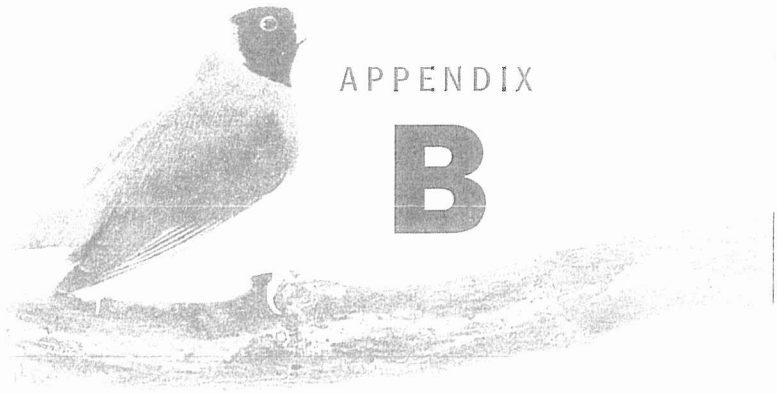
Every Java class has a single parent class. Java does not have the public and private class derivations of C++. Java class derivation is always the same. Java has

an additional encapsulation mechanism (besides the class)—the package. Entities defined in classes that do not specify a visibility have package scope, which makes them visible to all other classes in the package. Only one class in a package can be public. Rather than having public, private, and protected clauses in class definitions, the individual entities in Java classes can be defined to be public, private, or protected. All methods defined for a class are defined in the class. All binding of method calls to methods in Java is dynamic, unless the method is defined to be final, in which case it cannot be overridden and dynamic binding serves no purpose.

Class variables and class methods are specified to be static. In the absence of the `static` reserved word, variables are instance variables and methods are instance methods.

An interface defines the protocol of a class, but contains no variable definitions or method definitions. Interfaces are used to provide some of the benefits of multiple inheritance without all of the complexity of multiple inheritance. A class that implements an interface provides definitions for the methods of the interface.

Exception handling in Java is similar to that of C++, except that only objects of classes that descend from the predefined class `Throwable` can be exception objects. Propagation of exceptions is simpler in Java than it is in C++. The `throws` clause of Java is related to the `throw` clause of C++, but not closely. In Java, an exception class that appears in a `throws` clause means that the method in which `throws` appears can throw exceptions of that class or any of its descendants. A method cannot declare more exceptions in its `throws` clause than the method it overrides. A method that calls a method that can throw a particular exception must either catch and handle the exception, catch the exception and throw an exception that is declared in its `throws` clause, or declare the exception in its `throws` clause.



APPENDIX

B

Named Colors and Their Hexadecimal Values

The actual colors can be viewed at the following address:

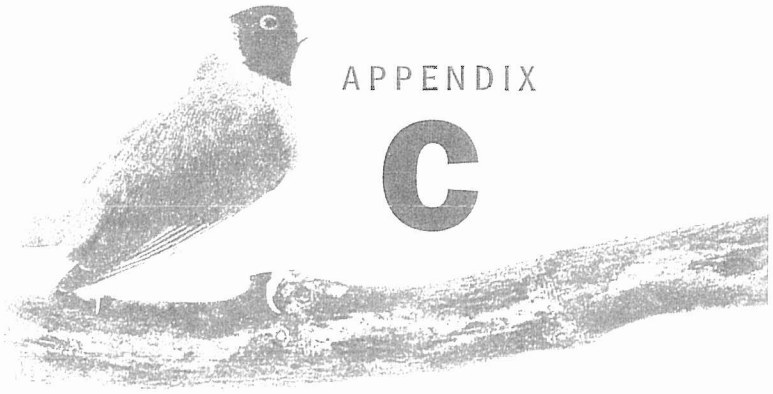
`http:/www.w3schools.com/html/html_colornames.asp`

Name	Hex Code	Name	Hex Code
aliceblue	F0FBFF	brown	A52A2A
antiquewhite	FAEBD7	burlywood	DEB887
aqua	00FFFF	cadetblue	5F9EA0
aquamarine	7FFFD4	chartreuse	7FFF00
azure	F0FFFF	chocolate	D2691E
beige	F5F5DC	coral	FF7F50
bisque	FFE4C4	cornflowerblue	6495ED
black	000000	cornsilk	FFF8DC
blanchedalmond	FFEBCD	crimson	DC143C
blue	0000FF	cyan	00FFFF
blueviolet	BA2BE2	darkblue	000088

Name	Hex Code	Name	Hex Code
darkcyan	008B8B	gold	FFD700
darkgoldenrod	B8860B	goldenrod	DAA520
darkgray	A9A9A9	gray	808080
darkgrey	A9A9A9	grey	808080
darkgreen	006400	green	008000
darkkhaki	BDB76B	greenyellow	ADFF2F
darkmagenta	8B008B	honeydew	F0FFF0
darkolivegreen	556B2F	hotpirik	FF6984
darkorange	FF8C00	indianred	CD5C5C
darkorchid	9932CC	indigo	4G0082
darkred	8B0000	ivory	FFFFFO
darksalmon	E9967A	khaki	FDE68C
darkseagreen	8FBCBF	lavender	E6E6FA
darkstateblue	483D8B	lavenderblush	FFFOF5
darkstategray	2F4F4F	lawngreen	7CFC00
darkstategrey	2F4F4F	lemonchiffon	FFFACD
darkturquoise	00CED1	lightblue	ADD8E6
darkviolet	9400D3	lightcoral	F08080
darkpink	FF1493	lightcyan	EOFFFF
darkskyblue	00BFFF	lightgoldenrodyellow	FAFAD2
dimgray	696969	lightgray	D3D3D3
dimgrey	696969	lightgrey	D3D3D3
dodgerblue	1E90FF	lightgreen	90EE90
firebrick	B22222	lightpink	FFB6C1
floralwhite	FFFAF0	lightsalmon	FFA07A
forestgreen	228B22	lightseagreen	20B2AA
fuchsia	FF00FF	ligthskyblue	87CEFA
gainsboro	DCDCDC	lightslategray	778899
ghostwhite	F8F8FF	lightslategrey	778899

Name	Hex Code	Name	Hex Code
lightsteelblue	B0C4DE	palegreen	98FB98
lightyellow	FFFFE0	paleturquoise	AFEEEE
lime	00FF00	palevioletred	D87093
limegreen	32CD32	papayawhip	FFEFD5
linen	FAF0E6	peachpuff	FFDAB9
magenta	FF00FF	peru	CD853F
maroon	800000	pink	FFC0CB
mediumaquamarine	66CDAA	plum	DDA0DD
mediumblue	0000CD	powderblue	B0E0E6
mediumorchid	BA55D3	purple	800080
mediumpurple	9370D8	red	FF0000
mediumseagreen	3CB371	rosybrown	BC8F8F
mediumslateblue	7B68EE	royalblue	4169E1
mediumspringgreen	00FA9A	saddlebrown	8B4513
mediumturquoise	48D1CC	salmon	FA8072
mediumvioletred	C71585	sandybrown	F4A460
midnightblue	191970	seagreen	2E8B57
mintcream	F5FFFA	seashell	FFF5EE
mistyrose	FFE4E1	sienna	A0522D
moccasin	FFE4B5	silver	C0C0C0
navajowhite	FFDEAD	skyblue	87CEEB
navy	000080	slateblue	6A5ACD
oldlace	FDF5E6	slategray	708090
olive	808000	slategrey	708090
olivedrab	6B8E23	snow	FFFAFA
orange	FFA500	springgreen	00FF7F
orangered	FF4500	steelblue	4682B4
orchid	DA70D6	tan	D2B4BC
palegoldenrod	EEE8AA	teal	008080

Name	Hex Code	Name	Hex Code
thistle	D8BFD8	white	FFFFFF
tomato	FF6347	whitesmoke	F5F5F5
turquoise	40E0D0	yellow	FFFF00
violet	EE82EE	yellowgreen	9ACD32
wheat	F5DEB3		



APPENDIX

C

Java Applets

C.1 Introduction
C.2 The Primary Applet Activities
C.3 The `paintComponent` Method
C.4 The `<object>` Tag
C.5 Applet Parameters
C.6 Simple Graphics
C.7 Color
C.8 Interactive Applets
Summary • *Review Questions* • *Exercises*

Applets provide another way of supporting computation in Web documents. Applets can provide interactivity and dynamic content through graphical user interface (GUI) components, as well as graphics and computation. This appendix is most accessible if the reader is already familiar with the Java programming language. Appendix A provides a brief introduction to Java for those who are already conversant in C++ and object-oriented programming. However, for those without that background, the complexity of Java and object-oriented programming is a formidable obstacle to gaining an understanding of applets in an appendix as brief as this one.

This appendix begins by providing an overview of applets and their relationship to XHTML documents. Then it describes the primary activities of applets, including initialization, starting execution, and stopping execution. Following this, attention turns to the `paintComponent` method, which is used to draw text and graphics both from a Java application to a display panel and from an applet to a browser screen. Next, the appendix describes the `<object>` tag,

which connects an XHTML document to an applet, and its attributes. XHTML documents can only refer to applet files because applets cannot be directly embedded in documents, unlike scripts in languages such as JavaScript. Then the appendix covers the technique for passing parameters from an XHTML document to an applet. Following this, it describes the methods of the `Graphics` class for drawing various figures and discusses how color can be used for the output of the `paintComponent` method.

Finally, interactive applets are discussed. This requires a description of how GUI components are defined in Java and how the Java event model is used to allow those components to provide interactivity.

Java 1.1 included the Abstract Windowing Toolkit (AWT), which provided basic drawing capabilities, as well as support for GUI components and an event model to allow users to interact with programs through them. Java 1.2 introduced the Swing package, which has a different set of GUI components that are similar in appearance to those of AWT but are very different internally. This appendix covers the AWT drawing facilities and the Swing GUI components. Be aware that the discussion of AWT graphics, event handling, and Swing components is brief and covers only a small portion of what exists.

All of the Java system software can be obtained from `http://java.sun.com`.

C.1 Introduction

Applets are Java programs whose execution is controlled in a way that is quite different from that of Java application programs. The purpose of an applet is related to that of client-side JavaScript: to provide processing capability and interactivity for XHTML documents.

When the browser encounters an applet in an XHTML document, it downloads the compiled version of the applet class (the `.class` file), along with any classes that the applet uses, from the server. Then an instance of the applet class is implicitly created and executed on the browser. (A more detailed description of these processes is given in Section C.2.) Because applets can include most of the Java language features, including widgets and event handling, they allow interactivity to be included in a document.

All applets must have some standard operations. The protocols for these common operations are gathered as method definitions in a predefined class named `JApplet`. All user-defined applets are written as subclasses of `JApplet`. Some of the methods inherited from `JApplet` are routinely overridden by user-defined applets.

The class header of an applet has the following form:

```
public class class_name extends JApplet { ... }
```

Although applets are good for graphics, they usually also display text and images. Rather than writing normal text to the screen, however, they must do this with a method that draws text into the browser display. All of the graphics

and text display capabilities used in applets can also be used in Java application programs.

Learning to write simple applets requires the following: First, you must understand how applets are specified in XHTML documents and how documents and applets interact. Second, you must learn how the relevant graphics library classes are used to display text and graphical figures. Third, you must see how the methods of `JApplet` are used, often in overridden versions, to control applet execution within the operation of the browser.

The simplest way to test an applet is with the Sun Microsystems program `appletviewer`, which enables you to see what an applet does without using a browser and a server. This technique has the advantage of simplicity, but it lacks some of the reality of using a Web browser. Of course, it is not much more complicated to use a browser with local XHTML documents and applets.

When Java applets appeared, they provided the first way to include client-resident computational capability in an XHTML document. JavaScript and its close relatives, AppleScript and VBScript, provide much of what the first release of Java provided through its applets. The power and versatility of Java—and especially its classes to support graphical user interfaces—have grown considerably since its original release. Furthermore, Java now has a large collection of class libraries that provide far more descriptive power than the scripting languages.

Applets are the second technology discussed in this book that provides computational capabilities for XHTML documents. Both JavaScript scripts and Java applets are interpreted on the client by the browser, which is good for server efficiency. JavaScript is both simpler to learn and simpler to use than Java. On the other hand, Java is much more expressive than JavaScript, especially because of the extensive array of class libraries now available. Furthermore, Java is faster than JavaScript, so if anything beyond short and simple computations is required, Java has the advantage.

Another area in which there is an advantage for Java applets over JavaScript is graphics. JavaScript includes virtually no graphics capability. So, even the graphics available in Java 1.1 (an early version of Java) are far superior to anything available in JavaScript. On the other hand, JavaScript has an advantage over Java because it is directly embedded in XHTML documents, whereas applets must be downloaded from the server when needed.

By 2002 the initial excitement regarding applets had diminished significantly, and other technologies, particularly scripting languages, had taken over many of the former uses of applets. The decrease in popularity of applets was in part due to the inconsistency of support from the browser vendors. The Netscape 4 browsers included Java virtual machines (intermediate code interpreters) that supported only version 1.1 of Java. (Version 1.2 of Java, which included many significant changes, was released in late 1998.) So, applets either had to be written using obsolete Java, or they would not be viewable on Netscape's browsers.[1] Up-to-date Java virtual machines were available from Sun

1. Netscape 6 and 7 include up-to-date Java interpreters.

for these browsers, but it was a sizable download, especially for those without high-speed Internet access, so many did not bother getting it. More recently, Microsoft stopped including support for Java in their browsers, although it can be downloaded from Sun. So, for awhile, Netscape clients were required to download Java virtual machines for their browsers, and Microsoft clients now are required to download them for their browsers. Because of these problems, as well as the availability of alternative technologies, many Web sites stopped creating applets and gradually eliminated the applets they were providing.

Java has become more heavily used on the server side in the form of servlets (discussed in Chapter 10, "Servlets and Java Server Pages") than on the client side in the form of applets. However, for now at least, Java applets are still included in some legacy Web sites, and some applets are still being written. Therefore, it is worthwhile for Web professionals to be familiar with them.

C.2 The Primary Applet Activities

Applets must include four fundamental methods through which the browser controls their execution. All of these are inherited from `JApplet`. When an XHTML document is being interpreted and displayed by a browser and an applet is encountered, the applet class code is downloaded and instantiated. Then the browser calls the applet's `init` method, which is inherited from `JApplet` but is often overridden in the user-defined applet class. The purpose of `init`, naturally, is to allow the applet to do some initialization. For example, if the applet has user-interface components (widgets), they are normally created in `init`. Upon return from the `init` method, the browser calls the applet method `start`, which begins execution of the applet's code. The `start` method is also implicitly called when the browser user returns to a document after viewing some other document. When the browser user directs the browser to follow a link from the current page to some new page, the browser calls the applet method `stop`. When the browser is stopped by the user, it calls the applet method `destroy`, which is used to do any cleanup that might be required at the end of the applet's life.

An applet's display is actually a frame, which is a multilayered structure. We are interested in just one of those layers, the *content pane*. The content pane is where applets put their output. Note that applets do not draw anything directly in the content pane; rather, applets draw in a panel and then add the panel to the content pane. This is also true for graphics in the Java applications world. For applications, a frame is created, and the filled panel is added to that frame's content pane. For applets, the filled panel is added to the applet's content pane.

There are two distinct categories of graphics operations for applets. In one category, something is drawn using a small set of primitive drawing methods. This use of primitives, which is sometimes called *custom drawing*, is done in overridden versions of the `paintComponent` method. Custom drawing must be done outside the subclass of `JApplet`. Typically, a subclass of `Jpanel` is created and used for custom drawing. Then the applet creates an instance of the

panel subclass and adds it to its content pane. The other category of graphics is the use of predefined graphics objects. Predefined graphics objects do not require the use of a paintComponent method. This means they can be placed directly in a panel that is created in the applet and then added to the applet's content pane. This is a simpler process.

C.3 The paintComponent Method

As previously stated, custom painting is done with the paintComponent method. However, when paintComponent is used in an applet environment, only the browser calls it; it should never be called by user code. paintComponent takes a single parameter: an object of class Graphics, which is defined in the java.awt package. This object, which is created by the browser, provides a collection of methods for drawing text and graphics. In a sense, the Graphics object provides a graphics context for paintComponent, similar to the device context for Windows and the graphics context in X-11. The methods of Graphics must be called through the browser-generated Graphics object. The protocol of the paintComponent method is as follows:

```
public void paintComponent(Graphics grafObj) { ... }
```

Initially, we discuss creating text with paintComponent. This is done with a method named drawString, which takes three parameters: a String literal, the *x* coordinate of the left end of the string, and the *y* coordinate of the base of the string. These coordinates are given in pixels.

Before calling paintComponent, the paintComponent method of the parent class (referred to with super) is called to paint the background of the display panel.

The following is an applet that displays a welcome message. It defines a subclass of JPanel that overrides paintComponent. The overriding paintComponent method draws the message with drawstring. The applet itself (the subclass of JApplet) creates a content pane, which is a Container object, with the getContentPane method. It also instantiates the subclass of JPanel, MessagePanel, by calling its constructor in a new clause. The init method, inherited from JApplet and overriden in the example, has just one statement, which adds the panel to its content pane by sending the MessagePanel object (created by the applet) to the add method of the Container object.

```
/* Wel.java
   An applet to illustrate the display of a string
   */
import java.applet.*;
import javax.swing.*;
import java.awt.*;
```

```
// The Wel applet
public class Wel extends JApplet {

// Create a content pane and the panel
   Container messageArea = getContentPane();
   MessagePanel myMessagePanel = new MessagePanel();

// The init method, which adds the panel to the applet
   public void init() {
      messageArea.add(myMessagePanel);
   }
}

// The panel class on which the message is painted
class MessagePanel extends JPanel {
   public void paintComponent(Graphics grafObj) {
      super.paintComponent(grafObj);
      grafObj.drawString("Welcome to my home page!", 50,
                         50);
   }
}
```

The call to `drawString` in `Wel` uses default values for the font parameters to display its string parameter. These parameters can be changed. The `Font` class, which is defined in `java.awt.Font`, has three variables that specify a font name, style, and size. Objects of the `Font` class can be created and used to set the instance variables in the `paintComponent` method. The font names and styles that are available depend on the implementation. We assume here that Times Roman and Courier are available in plain, boldface, and italic styles. The font styles are specified by named constants in the `Font` class, `PLAIN`, `BOLD`, and `ITALIC`. Objects of class `Font` are initialized through the three parameters in its constructor that specify the font name, style, and size. For example, consider the following instantiations:

```
Font font1 = new Font ("TimesRoman", Font.PLAIN, 36);
Font font2 = new Font ("Courier", Font.ITALIC, 24);
```

The font member of a `Graphics` object is set with the method `setFont`, which takes the `Font` object as its parameter.

The following is a revision of the `Wel` applet that uses a specific font, style, and size to display the same message as the earlier version:

```
/* Wel2.java
   An applet to illustrate the display of a string
   in a specific font, font style, and font size
   */
import java.applet.*;
import javax.swing.*;
import java.awt.*;

// The panel class on which the message will be painted
class MessagePanel extends JPanel {
   Font myFont = new Font("TimesRoman", Font.ITALIC, 24);

   public void paintComponent(Graphics grafObj) {
      super.paintComponent(grafObj);
      grafObj.setFont(myFont);
      grafObj.drawString("Welcome to my first home page!", 50,
                         50);
   }
}

// The Wel2 applet
public class Wel2 extends JApplet {

// The init method - create the content pane, instantiate
//   the message panel and add it to the content pane
   public void init() {
      Container messageArea = getContentPane();
      MessagePanel myMessagePanel = new MessagePanel();
      messageArea.add(myMessagePanel);
   }
}
```

C.4 The <object> Tag

The <object> tag is used to reference an applet in an XHTML document. The <object> tag is similar to the <img> tag used to specify images. The purpose of both <object> and <img> is to create a space in the document display where something can be put. In the case of <object>, when used for an applet, that something is whatever the applet paints. The form of the <object> tag and the attributes it uses for applets is as follows:

```
<object codetype = "application/java"
        classid = "java:applet_class_file"
        width = "applet display width"
        height = "applet display height">
</object>
```

The applet file is the compiled `.class` file. The width and height specify the size in pixels of the area in which the applet will paint.

The following XHTML document defines a simple XHTML document that uses the `Wel2` applet from Section C.3.

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- wel2.html
     A document to test the Wel2 applet
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Wel2 </title>
  </head>
  <body>
    <p>
      <object codetype = "application/java"
              classid = "java:Wel2.class"
              width = "500"
              height = "100">
      </object>
    </p>
  </body>
</html>
```

Unfortunately, there is a portability problem with the object element, even though it is part of the HTML 4.0 standard. Although Internet Explorer 6 (IE6) recognizes the `<object>` tag, it does not recognize the `classid` attribute. Instead, it uses the `code` attribute, which was associated with the deprecated `<applet>` tag. Also, if the `code` attribute is used in an object element, the `"java:"` part of the value for the `code` attribute must be dropped. The Netscape 7 (NS7) browsers do not recognize the `code` attribute. Oddly, Sun Microsystem's `appletviewer` also requires the nonstandard attribute `code`.

Figure C.1 shows a display of the output of `wel2.html`.

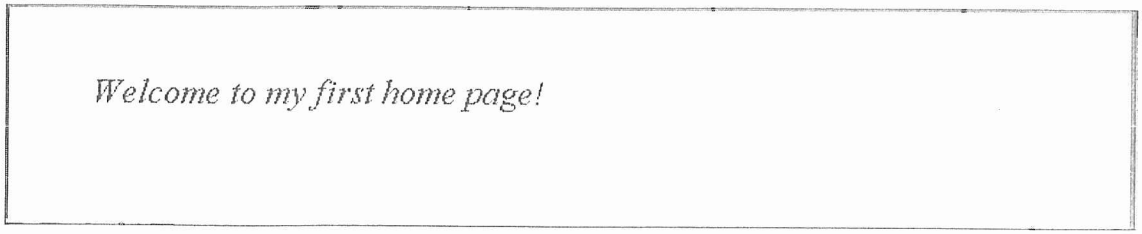


Figure C.1 A display of `Wel2.html`

C.5 Applet Parameters

A user can pass parameters to Java applications through the command line to the `main` method. Because applets do not have a `main` method, this obviously will not work for them. Still, it is convenient to be able to parameterize applets, and in fact, parameters can be sent to an applet from the XHTML document that calls it. This is done with the `<param>` tag, which is followed by a pair of named attributes. The first attribute is `name`, to which any name you like can be assigned. The second attribute is `value`, to which is assigned the value that you want to pass to the applet. For example, you might want to pass the size of an applet display element through a parameter, as shown in the following example:

```
<param name = "size"
      value = "24">
```

The applet uses the `getParameter` method to get the passed parameter value. It takes as its single parameter the name of the parameter, as a `String` literal. For example, to get the `size` parameter specified previously, the following could be used:

```
String mySize = getParameter("size");
```

If `getParameter` is called but the XHTML document did not specify the requested parameter value, `null` is returned. This provides a mechanism for specifying default values for such parameters. For example, consider the following:

```
int mySize;
String pString = getParameter("size");
if (pString == null)
  mySize = 36;
...
Font myFont = new Font("TimesRoman", Font.ITALIC, mySize);
```

The parameter value returned from `getParameter` is a `String` object. If it is actually an integer value (rather than a string), it must be converted to an `int` value. This can be done with the `Integer.parseInt` method. So, the previous `if` statement must have an `else` clause to do this conversion, as shown in the following example:

```
if (pString == null)
   mySize = 36;
else
   mySize = Integer.parseInt(pString)
```

The code to get parameters should appear in the `init` method. The following is a complete applet that does what the applet `Wel2` in Section C.3 did, except that the size of the displayed string is a parameter that can be specified from the XHTML document:

```
/* Wel3.java
   An applet to illustrate parameters
   */
import java.applet.*;
import javax.swing.*;
import java.awt.*;

// The panel class on which the message will be painted
class MessagePanel2 extends JPanel {
   Font myFont = new Font("TimesRoman", Font.ITALIC,
                          Wel3.mySize);

   public void paintComponent(Graphics grafObj) {
      super.paintComponent(grafObj);
      grafObj.setFont(myFont);
      grafObj.drawString("Welcome to my home page!", 50, 50);
   }
}

// The Wel3 applet
public class Wel3 extends JApplet {
   static int mySize;

   public void init() {
      Container messageArea = getContentPane();
      String pString;

// Get the fontsize parameter
      pString = getParameter("size");
```

```
// If it's null, set the size to 30; otherwise, use the
//  parameter value
      if (pString == null)
         mySize = 30;
      else mySize = Integer.parseInt(pString);

// Instantiate the panel with the message and add it to
//  the content pane
      MessagePanel2 myMessagePanel = new MessagePanel2();
      messageArea.add(myMessagePanel);
   }
}
```

The XHTML document that tests `Wel3` follows:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- wel3.html
     A document to test the Wel3 applet
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Wel3 </title>
  </head>
  <body>
    <p>
      <object codetype = "application/java"
              classid = "java:Wel3.class"
              width = "500"
              height = "100">
        <param name = "size"
               value = "40" />
      </object>
    </p>
  </body>
</html>
```

C.6 Simple Graphics

The `Graphics` class that the `Wel2` and `Wel3` applets used to put text into a document also includes methods for drawing lines, rectangles, and ovals. Although the Swing package is now used for creating GUI components, the

basic graphics capabilities of Java remain in the AWT package, which is where `Graphics` is defined. This section describes the basic drawing methods of AWT.

C.6.1 The Coordinate System

The methods that draw lines, rectangles, ovals, and arcs require the user to specify the location of those figures. Such locations are specified in terms of the `Graphics` coordinate system, which has the origin at the upper-left corner. This is exactly like the JavaScript coordinate system for positioning elements.

C.6.2 Lines

The `drawLine` method takes four parameters that specify the locations of the two ends of the line. For example, if the following `paintComponent` method were called, the call to `drawLine` would draw a line from the location (20, 10) to the location (60, 80).

```
public void paintComponent(Graphics grafObj) {
  grafObj.drawLine(20, 10, 60, 80);
}
```

C.6.3 Rectangles

The `Graphics` class provides methods for drawing rectangles and rectangles with rounded corners, where either of these can be filled or not filled. Ordinary (nonrounded corners) rectangles are drawn with either of these two methods:

```
drawRect(x1, y1, width, height)
fillRect(x1, y1, width, height)
```

In both cases, the location (`x1`, `y1`) specifies the upper-left corner of the rectangle, and the other two parameters specify the lengths of the rectangle's sides in pixels.

Specifying rectangles with rounded corners requires two more parameters in the method calls: one to specify the number of horizontal pixels in the rounding and one to specify the number of vertical pixels. If these two parameters are equal, the rounding is symmetric. The names of these two methods are `drawRoundRect` and `fillRoundRect`.

The following applet, `Rectangles.java`, draws the four rectangles shown in Figure C.2.

```
/* Rectangles.java
   An applet to illustrate drawing rectangles
   */
import java.applet.*;
import java.awt.*;
import javax.swing.*;

// The panel class for drawing
class MyPanel extends JPanel {

   public void paintComponent(Graphics grafObj) {
      super.paintComponent(grafObj);
      grafObj.drawRect(10, 10, 80, 60);
      grafObj.fillRect(120, 10, 60, 80);
      grafObj.drawRoundRect(10, 120, 80, 60, 20, 30);
      grafObj.fillRoundRect(120, 120, 60, 80, 40, 40);
   }
}

// The Rectangles applet
public class Rectangles extends JApplet {
   Container rectangleArea = getContentPane();
   MyPanel newPanel = new MyPanel();

// The init method for the applet - adds the panel to
//  the content area of the applet
   public void init() {
      rectangleArea.add(newPanel);
   }
}
```

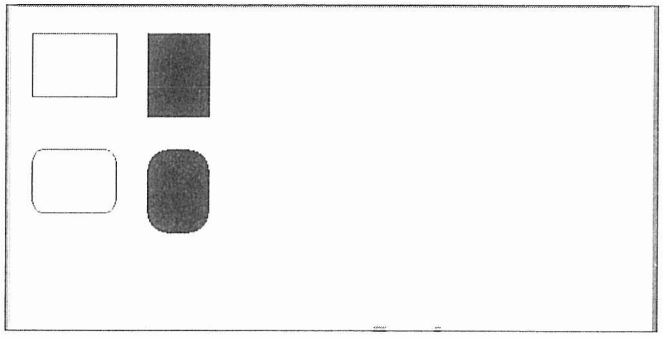


Figure C.2 A display of the output of the `Rectangles` applet

The `Graphics` class includes methods that draw so-called three-dimensional rectangles, which have shaded sides to make them appear like buttons that are either unpushed or pushed. Light shading on the left and upper sides makes a square look like an unpushed button. Dark shading on the left and upper sides makes a square look like a pushed button. The unpushed look is specified with a fifth parameter of `true`; the pushed look is specified with `false`. The name of the method for drawing these rectangles is `draw3DRect`.

C.6.4 Polygons

Polygons can be created by simply drawing a sequence of lines whose ends are connected. The points can be specified with two arrays, one consisting of the *x* coordinates and the other of the *y* coordinates. These two arrays, along with the number of points, are sent as parameters to the method `drawPolygon`. For example, an octagon could be drawn with the following applet, `Polygons`:

```
/* Polygons.java
   An applet to illustrate drawing a polygon
   */
import java.applet.*;
import java.awt.*;
import javax.swing.*;

// The panel for drawing
class PolyPanel extends JPanel {

   public void paintComponent(Graphics grafObj) {
      int xCoordinates [] = {30, 50, 64, 64, 50, 30, 16, 16, 30};
      int yCoordinates [] = {10, 10, 24, 44, 58, 58, 44, 24, 10};
      super.paintComponent(grafObj);
      grafObj.drawPolygon(xCoordinates, yCoordinates, 9);
   }
}

// The Polygons applet
public class Polygons extends JApplet {
   Container polyArea = getContentPane();
   PolyPanel newPanel = new PolyPanel();

// The init method, which adds the panel to the applet
   public void init() {
      polyArea.add(newPanel);
   }
}
```

Figure C.3 shows a display of the octagon drawn by the `Polygons` applet.

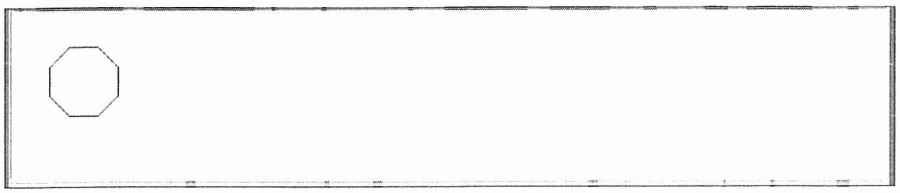


Figure C.3 A display drawn by the `Polygons` applet

An alternative technique for specifying a polygon is to create an object of class `Polygon`, which has a constructor with the same parameters as the `drawPolygon` method. Then an alternative version of `drawPolygon`, which takes a single `Polygon` parameter, can be called, as shown in the following example:

```
Polygon myPolygon = new Polygon(
                        xCoordinates, yCoordinates, 9);
grafObj.drawPolygon(myPolygon);
```

Polygons, like rectangles, can be filled by simply calling `fillPolygon` instead of `drawPolygon`.

C.6.5 Ovals

Drawing an oval is very similar to drawing a rectangle. In fact, the parameters to the oval-drawing methods are exactly those that could be sent to `drawRect`. The four parameters to the two oval-drawing methods, `drawOval` and `fillOval`, specify the coordinates of the upper-left corner and the width and height of the oval. A circle, of course, is just a "square" oval.

C.7 Color

In Java, specific colors are represented as objects of class `Color`. The `java.awt` package includes a collection of predefined `Color` objects that represent common colors, as well as methods for creating new colors and using colors in painting applets.

The Java abstract model of color uses 24 bits, with 8 bits for each of the three primary colors, red, green, and blue. The 13 predefined colors are shown in Table C.1, along with their RGB values.

Any color possible with the 24-bit specification can be constructed by creating an object of type `Color`, as shown in the following example:

```
Color myColor = new Color(x, y, z);
```

Table C.1 Predefined Java colors and their RGB values

Color Name	RGB Value
`Color.white`	`255, 255, 255`
`Color.black`	`0, 0, 0`
`Color.gray`	`128, 128, 128`
`Color.lightGray`	`192, 192, 192`
`Color.darkGray`	`64, 64, 64`
`Color.red`	`255, 0, 0`
`Color.green`	`0, 255, 0`
`Color.blue`	`0, 0, 255`
`Color.yellow`	`255, 255, 0`
`Color.magenta`	`255, 0, 255`
`Color.cyan`	`0, 255, 255`
`Color.pink`	`255, 175, 175`
`Color.orange`	`255, 200, 0`

Here, `x`, `y`, and `z` are integer values in the range of 0 to 255, representing the red, green, and blue components of the color.

The color of the `Graphics` object can be set with the `setColor` method, as follows:

```
grafObj.setColor(Color.magenta);
```

The background and foreground colors for a panel can be set with methods from the `Panel` class, as discussed in Section C.8.

C.8 Interactive Applets

A large part of the initial interest in Java was centered on applets, and a large part of this interest came from the possibility of making XHTML documents interactive. The support for interactivity in an applet is based on the reactive GUI components (widgets) that can be put in an applet display. This section describes how GUI components can be created in an applet and how user interactions with those components can be used to trigger computations. Because you have already learned about making GUI components react to user actions with XHTML and JavaScript, this section should be relatively easy to understand.

C.8.1 Java Swing GUI Components

The Swing package, defined in `javax.swing`, includes a collection of components that are what we have called *widgets*. A label component is an object of class `JLabel`. A `JLabel` object is a static string used to label other components. For example:

```
final JLabel lab1 = new JLabel("Customer name:");
```

A button is an object of class `JButton`. The parameter to the `JButton` constructor becomes the label in the button depiction:

```
JButton myButton = new JButton("Click me");
```

A checkbox is an object of class `JCheckbox`. The constructor for checkboxes has just one parameter, the label to appear next to the checkbox:

```
JCheckbox box1 = new JCheckbox("Hamburger");
JCheckbox box2 = new JCheckbox("French Fries");
JCheckbox box3 = new JCheckbox("Milk");
```

The `JCheckbox` constructor can include a second parameter, a Boolean. If `true` is sent to the constructor, the checkbox is initially checked; otherwise, it is initially unchecked.

Radio buttons are special buttons that are placed in a button group. A button group is an object of class `ButtonGroup`, whose constructor takes no parameters. The `JRadioButton` constructor, used for creating radio buttons, takes two parameters: the label and the initial state of the radio button (`true` or `false`). After the radio buttons are created, they are put in their button group with the `add` method of the group object. Consider the following example:

```
ButtonGroup payment = new ButtonGroup();
JRadioButton box1 = new JRadioButton("Visa", true);
JRadioButton box2 = new JRadioButton("MasterCard", false);
JRadioButton box3 = new JRadioButton("Discover", false);
payment.add(box1);
payment.add(box2);
payment.add(box3);
```

A text box is an object of class `JTextField`. The simplest `JTextField` constructor takes a single parameter, the length of the box in characters. For example:

```
JTextField name = new JTextField(32);
```

The `JTextField` constructor can also take a literal string, which is displayed as its contents. The string parameter, when present, appears as the first parameter.

Recall that the `paintComponent` method cannot paint an applet directly. Rather, it is used to paint an object of a subclass of `JPanel`, which is then added to the content pane of the applet. Components must also be added to a panel, but in this case it is a simple `JPanel` object that can be created in the applet class because the `paintComponent` method is not needed. The following code creates the panel object we use in the upcoming discussion of components:

```
JPanel myPanel = new JPanel();
```

The background color for a panel can be set with the `setBackground` method, as follows:

```
myPanel.setBackground(Color.yellow);
```

The `setForeground` method sets the default color of everything to be drawn in the panel. For example, the following statement changes the default drawing color to blue:

```
myPanel.setForeground(Color.blue);
```

After the components have been created with constructors, they must be placed in the panel with the `add` method, as shown in the following example:

```
myPanel.add(lab1);
```

Java defines several different objects called *layout managers* that determine how components are positioned in a panel. The default layout manager for Swing components is `BorderLayout`, which places components on the borders of the panel. This is fine for some situations. However, it is often convenient to have more control over where components are placed in a panel. For this, Java offers several alternative layout managers. One of these is `GridLayout`, which divides the panel area into rows and columns of compartments, each of which can contain a component. The parameters to the `GridLayout` constructor are the number of rows and columns, and the number of pixels between the rows and columns, respectively. The layout manager for a panel is specified with the `setLayout` method, which takes a layout manager object as its parameter. For example, consider the following code, which creates a new panel named `buttonPanel` and a `GridLayout` layout manager object for the panel:

```
JPanel buttonPanel = new JPanel();
buttonPanel.setLayout(new GridLayout(2, 3, 15, 15));
```

In this example, the panel's grid layout is specified to have two rows of three components each, with 15 pixels between the components.

The following example illustrates an applet that contains some simple GUI components. It uses a `GridLayout` manager object to put the components in a single column. Notice that the components are all placed in the panel object, which is ultimately added to the applet's content pane.

```
/* Pizza.java
   An applet to illustrate some GUI components with a pizza
   order form
   */
import java.awt.*;
import java.applet.*;
import javax.swing.*;

public class Pizza extends JApplet {
    Container contentPane = getContentPane();

    public void init() {

// Create a panel object and set its layout manager to put
// the components in a column
        JPanel myPanel = new JPanel();
        myPanel.setLayout(new GridLayout(20, 1, 10, 10));
        myPanel.setBackground(Color.cyan);

// Create a label for the form heading and add it to the panel
        Label myLabel = new Label("Pizza Order Form");
        myPanel.add(myLabel);

// Create a text field for the customer's name and
//  address and add them to the panel
        JLabel nameLabel = new JLabel("Name:");
        JTextField myName = new JTextField(30);
        JLabel addrLabel = new JLabel("Address:");
        JTextField myAddr = new JTextField(30);
        myPanel.add(nameLabel);
        myPanel.add(myName);
        myPanel.add(addrLabel);
        myPanel.add(myAddr);

// Create radio buttons for pizza size and add them to the panel
        JLabel sizeLabel = new JLabel("Pizza Size");
        ButtonGroup sizeGroup = new ButtonGroup();
        JRadioButton s1 = new JRadioButton("small");
        JRadioButton s2 = new JRadioButton("medium");
        JRadioButton s3 = new JRadioButton("large", true);
```

```
// Put the radio buttons in the button group
        sizeGroup.add(s1);
        sizeGroup.add(s2);
        sizeGroup.add(s3);

// Put the radio buttons in the panel
        myPanel.add(sizeLabel);
        myPanel.add(s1);
        myPanel.add(s2);
        myPanel.add(s3);

// Create checkboxes for toppings and add them to the panel
        JLabel topLabel = new JLabel("Toppings");
        Checkbox top1 = new Checkbox("sausage");
        Checkbox top2 = new Checkbox("pepperoni");
        Checkbox top3 = new Checkbox("extra cheese");
        Checkbox top4 = new Checkbox("hamburger");
        Checkbox top5 = new Checkbox("olives");
        Checkbox top6 = new Checkbox("mushrooms");
        myPanel.add(topLabel);
        myPanel.add(top1);
        myPanel.add(top2);
        myPanel.add(top3);
        myPanel.add(top4);
        myPanel.add(top5);
        myPanel.add(top6);

// Now add the panel to the content pane
        contentPane.add(myPanel);

    }  // End of init()
} // End of the Pizza applet
```

Figure C.4 shows a browser display of the `Pizza` applet.

Figure C.4 Results of running the `Pizza` applet

C.8.2 The Java Event Model

GUI components are not of much value unless user interactions with them trigger computations. In JavaScript, user interactions with components create events that can be caught by event handlers, which provide the computations. In Java, a similar model is used. GUI components are event generators. In Java, event handlers are called *event listeners*. Event listeners are connected to event generators through *event listener registration*. Listener registration is done with a method of the class that implements the listener interface, as described later in this section. The panel object into which the components are placed can be the event listener for those components. Only event listeners that are registered for a specific event are notified when that event occurs.

An event generator tells a listener of an event by sending a message to the listener (in other words, by calling one of the listener's methods). The listener method that receives the message implements an event handler. To make the event-handling methods conform to a standard protocol, a Java interface is used. An interface prescribes standard method protocols but does not provide implementations of those methods. This protocol could be specified by forcing

the event generator to be a subclass of a class from which it would inherit the protocol. However, the `JApplet` class already has a superclass, and in Java, a class can have just one parent class. Therefore, the protocol must come from an interface. A class cannot be instantiated unless it provides definitions for all methods in the interfaces that it implements.

A class that needs to implement a listener must implement an interface for those listeners. There are many classes of events and listener interfaces. The event classes appear in two categories: semantic events and low-level events. Table C.2 lists a few of the most commonly used classes of events.

Table C.2 Event classes

Class Name	User Actions That Create the Event Object
Semantic Event Classes	
`ActionEvent`	Click a button, select from a menu or list, or press Enter in a text field
`ItemEvent`	Select a checkbox or list item
`TextEvent`	Change the contents of a text field or text area
Low-Level Event Classes	
`ComponentEvent`	Resize, move, show, or hide a component
`KeyEvent`	Press or release a key
`MouseEvent`	Depress or release a mouse button, or move the cursor into or out of the component
`MouseMotionEvent`	Change the position of the mouse cursor over the component
`FocusEvent`	Get or lose focus for a component

Each semantic event listener interface prescribes one method for the handler. The low-level interfaces have several different handler methods. The handler methods for the two most commonly used semantic events are shown in Table C.3.

Table C.3 Semantic event listener interfaces and their handler methods

Interface	Handler Method
`ActionListener`	`actionPerformed`
`ItemListener`	`itemStateChanged`

As stated previously, the connection of a component to an event listener is made with a method of the class that implements the listener interface. Event listener registration establishes this connection. For example, because `ActionEvent` is the class name of event objects created by user actions on buttons, the `addActionListener` method is used to register a listener for buttons. The listener for button events created in a panel in an applet could be implemented in the panel. So, for a button named `button1` in a panel named `myPanel` that implements the `ActionEvent` event handler for buttons, we would register the listener with the following statement:

```
button1.addActionListener(this);
```

Each event handler method receives an event parameter that provides information about the event. Event classes have methods, such as `getState`, to access that information. For example, when called through a radio button, `getState` returns `true` or `false`, depending on whether the button was on or off, respectively.

All the event-related classes are in the `java.awt.event` package, so it must be imported to any applet class that uses events.

The following sample applet, `RadioB`, illustrates the use of events and event handling to display dynamic content in an applet. This applet constructs radio buttons that control the font style of the contents of a text field. It creates a `Font` object for each of four font styles. Each of these has a radio button to enable the user to select the style. The applet then creates a text string, whose font style will be controlled by the user through the radio buttons. The event handler `itemStateChanged` determines which radio button is pressed, after being informed by the `ItemEvent` object that a change has been made in the radio buttons. Then it sets the font style of the text string accordingly.

```
/* RadioB.java
   An applet to illustrate event handling with interactive
   radio buttons that control the font style of a text field
   */
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
import javax.swing.*;

public class RadioB extends JApplet implements ItemListener {

// Make most of the variables class variables, because both init
// and the event handler must see them
```

```
    private Container contentPane = getContentPane();
    private JTextField text;
    private Font plainFont, boldFont, italicFont, boldItalicFont;
    private JRadioButton plain, bold, italic, boldItalic;
    private ButtonGroup radioButtons = new ButtonGroup();
    private JPanel myPanel = new JPanel();

// The init method is where the document is initially built
    public void init() {

// Set the background color of the panel
        myPanel.setBackground(Color.cyan);

// Create the fonts
        plainFont = new Font("Serif", Font.PLAIN, 16);
        boldFont = new Font("Serif", Font.BOLD, 16);
        italicFont = new Font("Serif", Font.ITALIC, 16);
        boldItalicFont = new Font("Serif", Font.BOLD +
                                  Font.ITALIC, 16);

// Create the test text string, set its font, and add it to the
// panel
       text = new JTextField("In what font style should I appear?",
                             30);
        myPanel.add(text);
        text.setFont(plainFont);

 // Create radio buttons for the fonts and add them to the panel
        plain = new JRadioButton("Plain", true);
        bold = new JRadioButton("Bold");
        italic = new JRadioButton("Italic");
        boldItalic = new JRadioButton("Bold Italic");
        radioButtons.add(plain);
        radioButtons.add(bold);
        radioButtons.add(italic);
        radioButtons.add(boldItalic);

// Register the event handlers to myPanel
        plain.addItemListener(this);
        bold.addItemListener(this);
        italic.addItemListener(this);
        boldItalic.addItemListener(this);

// Now add the buttons to the panel
        myPanel.add(plain);
```

```
        myPanel.add(bold);
        myPanel.add(italic);
        myPanel.add(boldItalic);

// Now add the panel to the content pane for the applet
        contentPane.add(myPanel);

    }  // End of init()

// The event handler
    public void itemStateChanged (ItemEvent e) {

// Determine which button is on and set the font accordingly
if (plain.isSelected())
    text.setFont(plainFont);
else if (bold.isSelected())
   text.setFont(boldFont);
else if (italic.isSelected())
   text.setFont(italicFont);
else if (boldItalic.isSelected())
   text.setFont(boldItalicFont);

    } // End of itemStateChanged
} // End of RadioB applet
```

The `RadioB` applet produces the screen shown in Figure C.5.

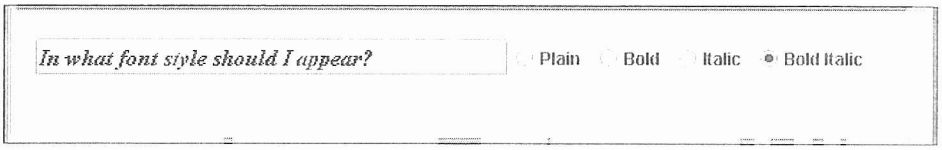


Figure C.5 The output of the `RadioB` applet

Summary

The impetus for Java's fast rise in popularity was the use of applets in constructing Web sites. An applet is a usually small collection of Java code defined in a class that is derived from `JApplet`. In objects of this class, a call is made to the `paintComponent` method, which can display a variety of things on the screen.

An XHTML tag, `<object>`, is used to inform the browser that an applet is to be downloaded from the server (or somewhere else) and run, creating some-

thing at that spot in the document. In a sense, applets are slaves to the XHTML document that calls them. The `<object>` tag also specifies the file type and the size of the area into which the applet will display and possibly some parameters to the applet.

The primary applet activities are `start`, which starts and restarts the applet; `stop`, which stops the applet's execution when the browser leaves the document that calls the applet; `destroy`, which is called when the user stops the execution of the browser; and `paintComponent`, which draws things on the screen. All of these methods are called by the browser. The `paintComponent`, `start`, and `stop` methods were described in this appendix.

Text and graphical objects are drawn by the applet with methods of the `Graphics` class. The coordinate system into which things are drawn has its origin in the upper-left corner. The `paintComponent` method takes a `Graphics` class object as a parameter. `paintComponent` calls other methods to produce the actual graphics in the applet area. The `drawString` method is used to display a character string. The `drawLine` method draws a line between two specified points. The `drawRect` and `fillRect` methods draw outlines of rectangles and filled rectangles, respectively. The `drawPolygon` and `fillPolygon` methods draw outlines of polygons and filled polygons, respectively.

Swing GUI components are implemented in Java as instantiations of their corresponding classes—for example, `JLabel` and `JButton`. The Java event model is related to that of JavaScript. Events are objects, often created by GUI components. Events are handled by methods called event listeners, which are connected to the GUI component objects. The listener object for events placed in a panel can be the panel itself. Event handlers can do virtually anything that can be done in an applet. One powerful possibility is that of dynamically changing the content of the document being displayed.

Review Questions

C.1 What three things happen when a browser finds a reference to an applet in an XHTML document?

C.2 From what class does `Applet` directly descend?

C.3 What are the two ways to test an applet?

C.4 What advantages do Java applets have over JavaScript?

C.5 What advantage does JavaScript have over applets?

C.6 What are the four fundamental methods used to control the basic operations of an applet?

C.7 How are the four fundamental methods for applet control called?

C.8 How is `paintComponent` called?

C.9 What parameter does `paintComponent` take and what is its origin?

C.10 What is the content pane of an applet?

C.11 Why is `paintComponent` used in a separate panel subclass?

C.12 Describe the parameters to `drawString`.

C.13 Describe the four required attributes of `<object>`.

C.14 Describe the attributes for `<param>`.

C.15 How are parameters from an XHTML document gotten into an applet that is referenced in that document?

C.16 Describe the parameters to `drawRect`.

C.17 How is the color of a `Graphics` object set?

C.18 How are radio buttons created in an applet?

C.19 What method places a GUI component into a panel?

C.20 Describe the parameters to a `GridLayout` constructor.

C.21 What is the event class for button clicks?

C.22 What is the event class for a checkbox selection?

C.23 What objects are sent notifications of GUI component events?

C.24 In what class is a method defined to register an event listener?

C.25 What event-handler method is used for button events?

Exercises

Write, test, and debug (if necessary) applets for the following specifications:

C.1 Modify the `Wel3` applet from Section C.5 to use parameters for the font and font style, as well as the font size. Test this applet with several different sets of parameters from the XHTML document that runs it.

C.2 Modify the `Wel3` applet to place the message inside an unfilled white circle that is centered in a filled blue square. The text must be black.

C.3 The applet for this exercise must display the Olympic logo, which consists of five overlaid circles. Below the logo must appear the text "The United States Olympic Committee." The circles must be blue, and the text must be red. The circles part of the logo must be enclosed in an unfilled green rectangle.

C.4 The applet for this exercise must display four checkboxes, labeled *Tacos*, *Chalupas*, *Burritos*, and *Nachos*. Beside each checkbox there must be a text

box labeled *Quantity*. This applet does not need to deal with events or event handling.

C.5 Modify the applet of Exercise C.4 by adding event handling for the input from the user. The applet must get a number from each text box, assuming 0 if the field is not changed. It must then compute the cost of the order, assuming that tacos cost $0.79, chalupas cost $1.19, burritos cost $1.39, and nachos cost $1.29.

C.6 The applet for this exercise must display two collections of six radio buttons, labeled *Red*, *Green*, *Blue*, *Yellow*, *Magenta*, and *Cyan*. The first collection must be labeled Foreground; the second must be labeled Background. These buttons must be implemented to control the foreground and background colors of the display. So that the foreground color can be seen, include a few geometric shapes and some text.

Index

Symbols and Numbers

-- (decrement), unary operator, 143, 315
(octothorpe), CSS, 101
(pound sign), Perl comments, 317
$ (dollar sign), PHP variables, 434
$ (scalar variables), in Perl, 312
% (hash variables), in Perl, 312
% (modulus), binary operator, 143, 315
%ENV, Perl hashes, 330
&& (AND operator)
  JavaScript, 155
  Perl, 320
  Ruby, 574
* (asterisk)
  SQL select clauses and, 525
  symbolic quantifier in pattern matching, 177
  universal selector, 101
* (multiplication), binary operator, 143, 315
** (exponentiation), binary operator, 315
@ (array variables), in Perl, 312
@import directive, CSS, 98
^ (circumflex) anchor, pattern matching and, 178
{ } (braces), block containers for Perl control statements, 321–322
+ (addition), binary operator, 143, 315
+ (plus), symbolic quantifier in pattern matching, 178
+ (plus), unary operator, 143
++ (increment), unary operator, 143, 315
< (less than), for Ruby subclasses, 594
< > (angle brackets)
  tag syntax and, 36
  XML syntax and, 269
= (assignment operator). *See* assignment operators
\ (back slashes), in URLs, 12–13
" (double quotes)
  Perl delimiters, 313–314
  PHP delimiters, 436
  Ruby delimiters, 564
/ (forward slashes), in URLs, 12–13
- (negate), unary operator, 143
! (NOT) operator
  JavaScript, 155
  Perl, 320
  Ruby, 574
? (question mark), symbolic quantifier in pattern matching, 178
' (single quotes)
  Perl delimiters, 313–314
  PHP delimiters, 436
  Ruby delimiters, 564
- (subtraction), binary operator, 143, 315
/ (division), binary operator, 143, 315
( ) (parentheses), forcing operator precedence with, 145
404 Not Found, HTTP response status codes, 18

A

<a> tag, for anchors, 52
absolute value, positioning elements, 231–234
Abstract Windowing Toolkit (AWT), 682, 692
Accept field, HTTP requests, 16
access control, Ruby, 592–594
action attribute, XHTML forms, 69
action body, JSP documents, 414
action buttons, XHTML forms, 77–78
action elements, JSP, 414, 422–426
action handlers, Ajax, 650
action methods, Rails, 610
ActionPack, Rails, 606
Active Server Pages (ASP), 491. *See also* ASP.NET
ActiveRecord, Rails, 606
actual parameters
  JavaScript, 169
  Perl, 333
  PHP, 454
  Ruby, 587
addEventListener method, DOM 2, 218–219
addition (+), binary operator, 143, 315
Adleman, Leonard, 20
Advanced Research Projects Agency (ARPA), 2
Ajax (Asynchronous JavaScript + XML), 637–658
  application example, 640–641
  controller method in Rails with Ajax, 655–657
  cross-browser support, 648–649
  exercises, 658
  form document, 641–643
  form document in Rails with Ajax, 650–652
  overview of, 27–28, 637–640
  Rails and, 607, 649–650
  Rails triggering, 652–655
  receiver phase, 646–648
  request phase, 643–645
  response document, 645–646
  review questions, 657–658
  summary, 657
alert method, JavaScript, 151–152
align attribute, XHTML tables, 65–66
alignment, CSS text, 116–117
American National Standards Institute (ANSI), 524

anchor characters, for pattern matching, 178
anchors tag (<a>), 52
AND (&&) operator
  JavaScript, 155
  Perl, 320
  Ruby, 574
angle brackets (< >)
  tag syntax and, 36
  XML syntax and, 269
angle operator (line input), Perl, 318
anonymous data types, XML schemas, 285
ANSI (American National Standards Institute), 524
Apache Group, 392, 432
Apache modules, 388
Apache server
  compared with IIS, 11
  extensions, 388
  overview of, 10–11
APIs (application programming interfaces)
  DOM as, 191
  for XML parsers, 303
Applet class, Java, 670
applets, Java, 681–708
  colors, 695–696
  event model, 701–705
  exercises, 707–708
  graphics, 691–695
  interactivity, 696
  JavaScript as alternative to, 135
  <object> tag, 687–689
  overview of, 24–25, 670, 681–684
  paintComponent method, 685–687
  parameters, 689–691
  primary activities of, 684–685
  review questions, 706–707
  summary, 705–706
  Swing package (javax.swing), 697–701
appletviewer, 683
application programming interfaces. *See* APIs (application programming interfaces)
applications
  Ajax, 640–641
  XML, 267
applications, Rails form
  controller and view, 617–621
  setting up, 615–616
  testing against database, 627–633
appName property, navigator object, 222
appVersion property, navigator object, 222
arithmetic operators
  C#, 484
  PHP, 437
ARPA (Advanced Research Projects Agency), 2
ARPAnet, 2–3
Array class, C#, 485
array construct, PHP, 446
Array methods, JavaScript, 166–168
array variables, Perl, 312
array_pop function, PHP, 450
array_push function, PHP, 450
arrays, Java, 665–666
arrays, JavaScript
  characteristics of, 164–165
  creating, 163–164
  methods, 166–167
  nested, 167–168
arrays, Perl
  associative. *See* hashes, Perl
  built-in array functions, 327
  built-in list functions, 327–328
  example, 328–329
  foreach statement, 326
  overview of, 324–326
arrays, PHP
  accessing array elements, 447
  converting strings to, 448
  creating, 446–447
  functions for dealing with, 447–453
  overview of, 445–446
  sequential access to, 449–451
  sorting, 451–453
arrays, Ruby
  built-in methods for arrays and lists, 580–583
  example, 583
  for-in statements, 579–580
  overview of, 578–579
ASCII code, 355
.asmx, 513
ASP (Active Server Pages), 491. *See also* ASP.NET
ASP.NET, 479–520
  code-behind files, 495–496
  coding control elements, 507
  control events, 504–505
  controls, 497
  document life cycle, 499–503
  documents, 493–495
  embedding programming code into XHTML documents, 25
  example, 508–510
  exercises, 519–520
  HTML controls, 497–499
  overview of, 491–493
  page-level events, 503
  response output for controls, 508
  review questions, 518–519
  server-side computation, 350
  summary, 517–518
  validation controls, 510–513
  Web controls, 505–507
  Web services, 513–516

ASP.NET documents
  life cycle, 499–503
  overview of, 493–495
assignment operators
  JavaScript, 149
  Perl, 317
  PHP, 440
  Ruby, 565, 574
associative arrays. *See* hashes
associativity rules
  JavaScript operators, 144, 156
  Perl operators, 315, 321
  Ruby operators, 566, 574
asterisk (*)
  SQL select clauses and, 525
  symbolic quantifier in pattern matching, 177
  universal selector, 101
asterisk parameters, 588
Asynchronous JavaScript + XML. *See* Ajax (Asynchronous JavaScript + XML)
attacks
  DoS (denial-of-service) attacks, 20
  SQL injection attacks, 628
attributes
  content spacing, 65–66
  events and, 196–199
  explicit attribute values, 88
  quoted attribute values, 88
  relational database entities, 522
  row or column labels, 63
  table content, 65–66
  tag syntax and, 36–37
attributes, Ruby, 593
attributes, XML
  complex types and, 287
  declaring, 275–276
  nested tags as alternative to, 270
  syntax of, 269
authentication, Web security and, 19
auto event wireup, ASP.NET, 503
`awk`, text processing languages, 312
AWT (Abstract Windowing Toolkit), 682, 692

B

`\b` (boundary), pattern matching and, 178
`<b>` tag (bold), 44
back slashes (\), in URLs, 12–13
background colors
  CSS, 115
  dynamic changes to, 241–242
  Java Swing components, 698
background images, CSS, 123–125
`background-position` property, CSS, 125
`background-repeat` property, CSS, 125
Backus-Naur form (BNF), 273
base types, XML schemas, 284
`belongs_to` relationships, in Rails, 625
Berners-Lee, Tim, 6, 34–35
binary entities, XML, 271
binary operators
  JavaScript, 143
  Perl, 315
binding, DOM to JavaScript, 192–193
binding, exceptions to handlers, 671–672
BITNET, 3
BizTalk, 304
block quotations, XHTML documents, 43–44
block tags, XHTML, 45
blocks, Ruby, 595–597
BNF (Backus-Naur form), 273
body
  functions, 168
  XHTML document structure, 38
`<body>`, handling events from body elements, 199–201
`bolder` values, `font-weight` property, 106
Boolean expressions, Ruby, 576
Boolean operators, PHP, 442
Boolean types
  JavaScript, 143
  PHP, 436
`border-color` property, CSS, 119
borders
  CSS box model, 118–120
  Java Swing components, 698
  XHTML tables, 61
`border-style` property, CSS, 118
`border-width` property, CSS, 118
box model, CSS, 118–123
  borders, 118–120
  margins and padding, 121–123
  overview of, 118
`<br />` tag, line breaks, 40–41
braces (`{ }`), block containers for Perl control statements, 321–322
`break` statements
  PHP, 443
  Ruby, 576, 577
browsers. *See* Web browsers
bubbling phase, DOM 2, 218
built-in functions, for arrays and lists
  Perl, 327–328
  Ruby, 580–583
bullets, list, 110–111
buttons
  handling events from button elements, 201–206
  Java Swing components, 697

by-reference parameters
C#, 486
JavaScript, 171
PHP, 455
bytecodes, Java, 24
by-value parameters
C#, 486
JavaScript, 170
PHP, 454

C
C language
C++ compared with, 662
case sensitivity, 139
Perl and, 331
C#
classes, 486–487
CLS restrictions and, 482
control statements, 485–486
data structures, 484–485
delegates, 488
file storage, 489–491
methods, 486–487
.NET languages and, 480–481
overview of, 481–483
primitive types and expressions, 483–484
program structure, 488–489
properties, 487–488
Ruby compared with, 561
structures, 486–487
C++
classes, 666
data types, 664
exception handling, 671
Java compared with, 24, 662–664
methods, 668
object variables, 664
Ruby compared with, 561
`throw` clause, 673
callbacks, Ajax, 643
`<caption>` tags, XHTML tables, 61
capturing phase, DOM 2, 217
the cascade, 128
cascading style sheets, 97. *See also* CSS (cascading style sheets)
Cascading Style Sheets-Positioning (CSS-P), 231
case constructs, Ruby, 575–576
case sensitivity
HTML vs. XHTML, 88
JavaScript, 139
Perl, 314
Ruby, 564
SQL, 525
`catch` clause, Java, 671
CDATA (character data), 272
`cellpadding` attribute, XHTML tables, 67–68
cells, table, 61
`cellspacing` attribute, XHTML tables, 67–68
CERN (Conseil European pour la Recherce Nucleaire), 6, 34
CGI (Common Gateway Interface)
`CGI.pm` module, 356–359
cookies, 377–381
exercises, 383–385
form example, 360–365
link to XHTML document, 351–354
overview of, 350–351
Perl and, 25, 311
query string format, 354–356
review questions, 382–383
servlets compared with, 388
summary, 381–382
survey example, 365–377
Web servers interacting with database systems via, 10
`cgi-bin`, Web servers, 351–352
`CGI.pm` module, 356–359
`cookie` function, 378–381
`header` function, 358–359
overview of, 356
shortcuts, 357–358
character classes, pattern matching in JavaScript, 176–178
character data (CDATA), 272
character entities. *See* entities
character patterns, pattern matching in JavaScript, 176–178
checkboxes
DOM addressing and, 194–195
form control types, 70
`<input>` tag, 71–72
Java Swing components, 697
JSTL, 424
checked exceptions, Java, 673
circumflex (^) anchor, pattern matching and, 178
class selectors, CSS, 100–101
classes
C#, 484–487
C++, 666
JavaScript pattern matching and, 176–178
servlets, 389–390
classes, ASP.NET
document class, 492–493
`HtmlControl` class, 499
`Random` class, 494–495
classes, Java
applets, 684, 691–696
events, 702

exceptions, 670–671
objects and methods and, 666–669
classes, Ruby
access control, 592–594
inheritance, 594–595
overview of, 590–592
classic syntax, JSP documents, 413
CLI (Common Language Interface), 481–482
`click` events, DOM button elements, 201–206
clients, Web, 7. *See also* Web browsers
client/server architectures
JDBC, 531–532
Microsoft Access, 530
overview of, 529–530
Perl DBI/DBD, 530–531
PHP, 531
client-side, JavaScript
dynamic XHTML documents and, 230
overview of, 134
uses of, 135–136
closing tags, HTML vs. XHTML, 88
closure, 646
CLR (Common Language Runtime), 481–483
as base technology for .NET, 481
overview of, 482–483
CLS (Common Language Specification), 482
Codd, E. F., 522
code, for control elements, 507
code blocks, Ruby, 577, 595–597
`<code>` tag, XHTML, 44
code-behind files
ASP.NET, 492–493, 495–496
example, 509
coercions
JavaScript, 146
PHP, 438–439
collection classes, C#, 484
`Color` class, Java applets, 695–696
`color` property
CSS, 115
XHTML documents, 241–242
colors
hexadecimal values, 677–680
Java applets, 695–696
colors, CSS
named colors, 114–115
properties, 115
property values, 103
`cols` attribute, `<frameset>` tag, 82–83
`colspan` attribute, XHTML tables, 63–65
COM
creating components, 480
PHP support for, 432
comments
ASP.NET, 494
CSS, 99
Java, 415
JavaScript, 139
Perl, 317
PHP, 434
XHTML, 37
XML, 269
Common Gateway Interface. *See* CGI (Common Gateway Interface)
Common Language Interface (CLI), 481–482
Common Language Runtime. *See* CLR (Common Language Runtime)
Common Language Specification (CLS), 482
COMMON LISP, 293
Common Type System (CTS), 482
comparison operators, JavaScript, 172–173
complete paths, URL paths, 13
complex data type, XML schemas, 284, 286–288
`complexType` tag, XML, 286
components, 480
compound statements, JavaScript, 154
computations
server-side, 350
XHTML and, 24–25, 682–683
concatenation. *See* string catenation operator
`confirm` method, JavaScript, 152
conflict resolution, CSS, 126–128
Conseil European pour la Recherce Nucleaire (CERN), 6, 34
`const` reserved word, C#, 484
constructors, JavaScript, 162, 174–175
constructors, servlet constructor for cookie objects, 400
content
dynamic changes to, 244–246
tag, 36
content pane, Java applets, 684
content-based style tags, XHTML, 44
`Content-length` field, HTTP requests, 16
contextual selectors, CSS, 100
control constructs
JavaScript, 154
Perl, 322
Ruby, 575
control events, ASP.NET, 504–505
control expressions
JavaScript, 155–156
Perl, 319–321
Ruby, 573–574
control statements
C#, 485–486
C++, 663
Java, 663
JSTL Core library, 414, 422–426
PHP, 442

control statements, JavaScript
control expressions, 155–156
overview of, 154
selection statements, 156
control statements, Perl
control expressions, 319–321
overview of, 319
selection and loop statements, 321–324
control statements, Ruby, 573–578
control expressions, 573–574
overview of, 573
selection and loop statements, 575–578
controller
in MVC architecture, 606
Rails, 617–621
Rails with Ajax, 655–657
controls, ASP.NET. *See* server controls
controls, XHTML tags for, 69
`Cookie` class, servlets, 400
`cookie` function, `CGI.pm` module, 378–381
cookies
overview of, 399–400, 469–470
PHP support for, 470–471
servlet support for, 400–401
session information stored in, 377–381
coordinate system, Java graphics, 692
Coordinated Universal Time (UTC), 613
copy mode, PHP processor, 433
CORBA
PHP support for, 432
RPC technologies, 304
core, JavaScript, 134
Core library, JSTL, 414, 422
`CREATE DATABASE` command, MySQL, 533
`CREATE TABLE` clauses, SQL, 528–529
`CREATE TABLE` command, MySQL, 533
CSNET, 3
CSS (cascading style sheets)
Ajax and, 639
background images, 123–125
borders, 118–120
box model, 118
class selectors, 100–101
colors, 114–115
conflict resolution, 126–128
exercises, 131–132
font properties, 104–110
font styles, 44
generic selectors, 101
`id` selectors, 101
list properties, 110–114
margins and padding, 121–123
overview of, 95–97
property value forms, 103–104
pseudo classes, 102–103
review questions, 130–131
simple selectors, 99
`<span>` and `<div>` tags, 125–126
style sheet levels, 97–98
style specification formats, 98–99
summary, 128–129
text alignment, 116–117
`text-decoration` property, 109–110
universal selectors, 101–102
XML documents displayed with, 291–292
CSS-P (Cascading Style Sheets-Positioning), 231
Ctrl key, selections with, 76
CTS (Common Type System), 482
`current` function, PHP arrays, 449
`currentTarget` property, DOM 2, 219
custom drawing, Java, 684

D

data structures
C#, 484–485
Java, 664–666
data types
C++, 664
Java, 664–666
MySQL, 533
numeric. *See* numeric literals (types)
primitive. *See* primitives
Ruby, 562
strings. *See* string literals
data types, PHP
Booleans, 436–437
double type, 435
integers, 435
scalar type conversions, 438–439
strings, 436
variables, 435
data types, XML schemas
complex types, 286–288
overview of, 284–285
simple types, 285–286
database access, 521–559
client/server architectures, 529–530
example, 537–538
exercises, 559
JDBC, 531–532
JDBC and My SQL, 548–556
joins, 526–527
JSTL Relational Database library, 414
Microsoft Access, 530
MySQL, 532–534
overview of, 521–522
Perl and MySQL, 535–537
Perl DBI/DBD, 530–531
PHP, 432, 531

PHP and My SQL, 539–547
relational databases, 522–524
review questions, 557–559
SQL and, 524–525
summary, 556–557
database driver (DBD), 530–531
database interface (DBI), 530–531, 535–537
databases, metadata in, 551–553
databases, Rails
building, 622–627
overview of, 622
testing application against, 627–633
data-driven model, XSLT, 294
`Date` object, JavaScript, 150
DBD (database driver), 530–531
DBI (database interface), 530–531, 535–537
DCOM, RPC technologies, 304
`<dd>` tag, XHTML definitions, 59–60
debuggers
JavaScript, 183
Perl, 312
declaration blocks, ASP.NET document, 494
declarations, DTD
attributes, 275–276
elements, 273–274
entities, 276
local vs. global, 285
overview of, 272
decrement (--), unary operator, 143, 315
definition lists, XHTML, 59–60
delegates, C#, 488
`DELETE` clauses, SQL, 528
`delete` operator, Perl hashes, 330
delimiters
Perl, 313–314
PHP, 436
Ruby, 563–564
denial-of-service (DoS) attacks, 20
Department of Defense (DoD), 2
deprecated features, HTML, 35
dereferencing, 331
derived data types, XML schema, 284
descendant selectors, CSS, 100
`DESCRIBE` command, MySQL, 533–534
diamond operator (line input), Perl, 318
`die` operator, Perl, 328
Diffie, Whitfield, 19
directives
ASP.NET, 494
CSS `@import` directive, 98
JSP, 414–415
Rails, 624
directories
Rails, 609–610
Web servers, 9
`display`, CSS style sheet, 292
`<div>` tags, CSS, 125–126
division (/), binary operator, 143, 315
`<dl>` tag, XHTML definition lists, 59–60
DNS (Domain Name System), 5
DOCTYPE
RFC 2616 (HTTP standard), 15
SGML, 37–38
Web standards, 34–35
XHTML document validation, 49–50, 51
document class, ASP.NET, 492–493
document entity, XML, 271
document level, CSS levels, 97–99
document life cycle, ASP.NET, 499–503
`Document` object, JavaScript, 151, 190
Document Object Model. *See* DOM (Document Object Model)
document root, in Web server directories, 9
document structure, XHTML
block quotations, 43–44
character entities, 45–46
fonts styles and sizes, 44–45
headings, 41–42
horizontal rules, 46
line breaks, 40–41
`meta` element, 46–47
overview of, 37–38
text markup, 38–40
whitespace, 41
document structure, XML, 271–272
document type definitions. *See* DTDs (document type definitions)
document types, MIME, 13–15
document validation, XHTML, 49–51
documents
ASP.NET, 493–495, 499–503
JSP, 413–415
units of information on Web, 6
virtual document trees, 10
XHTML. *See* XHTML documents
XML. *See* XML documents
XSL, 293
DoD (Department of Defense), 2
`doGet` method, `HttpServlet` class, 390
dollar sign ($), PHP variables, 434
DOM (Document Object Model), 191–193
Ajax and, 639
binding to JavaScript, 192–193
event models, 195
JavaScript and, 136
tree modification, 225
tree traversal, 224
tree-like structure of, 191–192
versions of, 191
XML processor building DOM tree, 303–304

DOM 0 event model, 195–216
  dragging and dropping elements, 257–262
  `focus` events, 207–209
  handling events from body elements, 199–201
  handling events from button elements, 201–206
  handling events from text box and password elements, 207
  overview of, 195–198
  validating form input, 209–216
DOM 2 event model, 217–222
  dragging and dropping elements, 257–262
  event propagation, 217–218
  example, 220–222
  overview of, 217
  registration, 218–220
DOM address, 193–194
Domain Name System (DNS), 5
domain names, 4–6
domains, 4
`doPost` method, `HttpServlet` class, 390
DoS (denial-of-service) attacks, 20
double quotes (")
  Perl delimiters, 313314
  PHP delimiters, 436
  Ruby delimiters, 564
double type, PHP, 435
double-precision floating-point form, 313
`do-while` statements
  JavaScript, 161
  PHP, 443
drag-and-drop, 257–262
Dreamweaver, Macromedia, 22
`DROP` clauses, SQL, 528
`<dt>` tag, XHTML definition terms, 59–60
DTDs (document type definitions)
  declaring attributes, 275–276
  declaring elements, 273–274
  declaring entities, 276
  disadvantages of and alternatives to, 281
  internal and external, 277–279
  overview of, 272–273
  sample DTD, 276–277
  SGML and, 37–38
  XHTML document validation, 51
  XML syntax and, 268
  XSLT using to format XML documents, 293
dynamic content, XHTML, 244–246
dynamic documents, Rails, 612–615
dynamic documents, XHTML, 229–264
  absolute positioning, 231–234
  color changes, 241–242
  dragging and dropping elements, 257–262
  dynamic content, 244–246
  exercises, 263–264
  font changes, 242–243
  mouse cursor location, 250–252
  moving elements, 236–239
  moving elements slowly, 254–257
  overview of, 229–230
  positioning elements, 230–231
  reacting to mouse clicks, 253–254
  relative positioning, 234–236
  review questions, 262–263
  stacking elements, 247–250
  static positioning, 236
  summary, 262
  visibility of elements, 239–240
dynamic types, PHP, 433

E

ECMA (European Computer Manufacturers Association), 134
ECMAScript, 134
EL (Expression Language)
  JSP Expression Language, 419–422
  JSTL Functions library, 414
element nesting, HTML vs. XHTML, 89
`element` tag, XML schemas, 285
element-only elements, 286
elements, JSP, 413, 416
elements, XHTML
  absolute positioning, 231–234
  accessing in JavaScript, 193–195
  dragging and dropping, 257–262
  dynamic content, 244–246
  moving, 236–239
  moving slowly, 247–250
  overview of, 21
  relative positioning, 234–236
  stacking, 247–250
  static positioning, 236
  tag syntax and, 36
  visibility of, 239–240
elements, XML
  declaring, 273–274
  declaring attributes, 275–276
  defined, 267
  `element` tag, 285
  element-only elements, 286
  `schema` root element, 282
`elseif` statement, PHP, 442
`elsif` statement
  Perl, 322
  Ruby, 575
`<em>` tag (emphasis), CSS, 44
*E*mbedded *Ruby* (ERb), 611
`encoding`, XHTML document structure, 37
encryption, 19

end-of-file (EOF) character
  Perl, 321
  Ruby, 574
entities
  character entities in XHTML document structure, 45–46
  declaring in XML, 276
  relational databases, 522
  XML document structure and, 271–272
enumerations, C#, 483, 485
environment variables, Perl, 330
EOF (end-of-file) character
  Perl, 321
  Ruby, 574
ERb (*E*mbedded *Ruby*), 611
errors
  in scripts, 181–183
  in XML documents, 277
escape sequences, Perl, 313–314
European Computer Manufacturers Association (ECMA), 134
event classes, Java, 702
event handlers
  defined, 196
  DOM 0 vs. DOM 2, 195
  dragging and dropping elements, 257–262
  handling events from body elements, 199–201
  handling events from button elements, 201–206
  handling events from text box and password elements, 207
  HTML controls, 504–505
  IE vs. Firefox, 250–251
  registration in DOM 2, 218–220
  validating form input, 209–216
event listener registration, Java, 701
event listeners, Java, 701–703
event models
  DOM 0. *See* DOM 0 event model
  DOM 2. *See* DOM 2 event model
  Java, 701–705
event processing, SAX, 303
event propagation, DOM 2, 217–218
event-driven compilation, JavaScript, 136
event-driven programming, DOM, 196
events
  defined, 196
  tag attributes and, 196–199
exception classes, Java, 670–671
exception handling, Java
  binding exceptions to handlers, 671–672
  example of exception handling, 674–675
  exception classes, 670–671
  exception handlers, 671
  exception propagation, 672–673
  `throws` clause, 673
`exists` operator, Perl hashes, 330
explicit attribute values, HTML vs. XHTML, 88
explicit type conversions
  JavaScript, 147–148
  PHP, 438–439
`explode` function, PHP arrays, 448
exponentiation (`**`), binary operator, 315
Expression Language (EL)
  Functions library in JSTL, 414
  JSP Expression Language, 419–422
expressions
  C#, 483–484
  PHP, 437
eXtensible Hypertext Markup Language. *See* XHTML (eXtensible Hypertext Markup Language)
eXtensible Markup Language. *See* XML (eXtensible Markup Language)
eXtensible Stylesheet Language (XSL), 293
external DTD, 272
external level, CSS, 97–99

F

facets, of base types, 284–285
`FALSE`, PHP Boolean types, 436
FCL (Framework Class Libraries), 482, 488
fields, relational databases, 522
file extensions, PHP, 432
file input, Perl, 317–318, 339–341
file management, PHP, 466–469
  file use indicators, 467
  locking files, 469
  opening/closing files, 466–467
  reading from files, 467–468
  writing to files, 469
file output, Perl, 317, 339–341
file pointers, PHP, 466
file storage, C#, 489–491
File Transfer Protocol (`ftp`)
  protocols running on top of TCP/IP, 5
  Web servers supporting, 10
file variable, PHP, 466
filehandles, Perl, 317–318, 339–341
files, referencing, 339–341
filters, converters for XHTML documents, 22–23
Firebug, 183
FireFox
  Ajax support, 648–649
  CSS standards and, 96
  debugger for JavaScript, 183
  event handlers, 250–251
  `navigator` object and, 223–224
  popularity of, 8

pseudo classes and, 102
XML support, 268
`Float` class, Ruby, 563
`float` property, for images and tables (CSS), 115–116
`flock` function, locking files in PHP, 469
flow-control. *See* control statements, JavaScript
`focus`, pseudo class, 102–103
`focus` events, JavaScript for XHTML documents, 207–209
`font-family` property, CSS, 104–105
fonts, CSS, 104–109
families, 104–105
overview of, 104
shorthand for, 106–108
sizes, 105
styles, 106
variants, 105
weights, 106
fonts, XHTML
dynamic changes to, 242–243
heading levels and sizes, 41–42
styles and sizes, 44–45
`font-size` property, CSS, 105
`font-style` property, CSS, 106
`font-variant` property, CSS, 105
`font-weight` property, CSS, 106
`fopen` function, opening files in PHP, 466
`for` statements
JavaScript, 159–161
Perl, 322–323
PHP, 443
`forEach` element, JSTL, 424
`foreach` statement
C#, 485
Perl, 326
PHP, 451
foreground color
dynamic changes to, 241–242
Java Swing components, 698
`for-in` statements
JavaScript loops, 161
Ruby arrays, 579–580
form data, 69
`<form>` tag
action buttons, 77–78
`method` attribute, 355
overview of, 69
formal parameters
JavaScript, 170
Perl, 333
PHP, 454
Ruby, 587
formats
CSS styles, 98–99
URLs, 11–12
XHTML documents, 38
XSLT objects, 295
forms
Ajax form document, 641–643
CGI (Common Gateway Interface), 360–365
PHP, 432, 460–466
Rails with Ajax form document, 650–652
forms, Rails
application setup, 615–616
controller and view for form application, 617–621
overview of, 615
forms, servlet
ballot form example, 401–407
survey example, 392–398
forms, XHTML, 69–81
action buttons, 77–78
example form, 78–81
`<form>` tag, 69
`<input>` tag, 69–74
overview of, 69
`<select>` tag, 74–76
`<textarea>` tag, 76–77
validating form input, 209–216
`forms` array, JavaScript, 190
forward slashes (/), in URLs, 12–13
`<frame>` tag, XHTML, 83
frames, 81–87
content of, 83–86
framesets, 82–83
nesting framesets, 86–87
overview of, 81–82
`<frameset>` tag, XHTML, 82
framesets
nesting, 86–87
overview of, 82–83
Framework Class Libraries (FCL), 482, 488
`fread` function, reading files in PHP, 468
`FROM` clause, SQL, 525
FrontPage, Microsoft, 22
`ftp` (File Transfer Protocol)
protocols running on top of TCP/IP, 5
Web servers supporting, 10
fully qualified domain names, 5
function definition
JavaScript, 168
Perl, 331–332
function headers, Perl, 331
functional-style programming languages, 293
functions, JavaScript, 168–169
function definition, 168
`moveText` function, 255
overview of, 137
functions, Perl
built-in array functions, 327
built-in list functions, 327–328

CGI.pm module of common functions, 356–359
cookie function, 378–381
example, 334–336
function definition, 331–332
overview of, 331–332
string functions, 316–317
functions, PHP
array management, 447–453
cookies, 470–471
explicit type conversion, 439
file management, 466–469
mysql_connect function, 540–541
output, 440–441
overview of, 453–454
parameters, 454–455
pattern matching, 457–460
predefined, 437
special characters, 539
stings, 438
variable lifetime, 457
variable scope, 455–457
functions, Rails, 617–618
functions, Ruby
built-in functions, for arrays and lists, 580–583
screen output in Ruby, 571
Functions library, JSTL, 414
fwrite function, writing to files in PHP, 469

G

garbage collection, Java, 662
Garrett, Jesse James, 640
Gates, Bill, 304
general entities, 276
generic fonts, CSS, 105
generic selectors, CSS, 101
GenericServlet class, 389
GET method
form methods, 69
HTTP requests, 16, 515
query strings and, 354–356
Web services and, 304
getElementById method, XHTML, 194
gets method, Ruby, 571
getter methods, Ruby, 593
getWriter method, HttpServlet class, 390
GIF (Graphic Interchange Format), 47–48
global declarations, DTDs, 285
global scope
JavaScript variables, 169
PHP variables, 456–457
Gmail, 640
Google Maps, 640
Gopher, Web servers supporting, 10
goto, C#, 485
grabber event handler, 257
Graphic Interchange Format (GIF), 47–48
graphical user interface (GUI), 681
graphics, Java, 691–695
coordinate system, 692
custom drawing, 684
JavaScript compared with Java, 683
lines, 692
ovals, 695
overview of, 691–692
paintComponent method, 685–687
polygons, 694–695
predefined graphics objects, 685
rectangles, 692–694
Graphics class, Java applets, 691–695
GUI (graphical user interface), 681

H

<h1-6> tags, 41–42
hackers, 20
Hansson, David Heinemeier, 27, 606
has_many relationships, in Rails, 625
hashes
Perl, 312, 329–330
Ruby, 583–585
head, XHTML document structure, 38
header, function, 168
header fields, HTTP requests, 16
headings, XHTML documents, 41–42
height attribute, XHTML images, 48
Hellman, Martin, 19
help boxes, 246
helper applications, 14
hexadecimal values, for named colors, 114–115, 677–680
Hibbs, Curt, 607
horizontal rules, XHTML document structure, 46
hostname field, HTTP requests, 16
hover, pseudo class, 102–103
<hr />, horizontal rules, 46
html
MIME types, 14
XML root element, 269
HTML (Hypertext Markup Language)
deficiencies in leading to creation of XML, 266–267
origins of, 34
syntactic differences between HTML and XHTML, 88–89
versions of, 34–35
XHTML compared with, 35–36
XHTML replaces, 21
HTML controls
event handlers for, 504–505
overview of, 497–499
Web controls compared with, 505–506
HtmlControl class, 499

htmlspecialchars function, PHP, 539
HTTP (Hypertext Transfer Protocol)
  for communication between browsers and Web servers, 8–9
  overview of, 15
  requests, 15–17, 351
  responses, 17–18, 353–354
  servlets and, 388
  stateless nature of, 399
  URL format, 12
httpd, 10
HttpServlet class, 389
HttpSession object, 408
HyperCard, Apple, 6
hypermedia, in Web documents, 6
hypertext, 6. *See also* links
Hypertext Markup Language. *See* HTML (Hypertext Markup Language)
Hypertext Transfer Protocol. *See* HTTP (Hypertext Transfer Protocol)

I

<i> tag, for italics, 44
id attribute
  HTML vs. XHTML, 89
  for target elements, 54–55
id selectors, CSS, 101
identifiers, JavaScript, 139
ids, DOM addresses, 194
IE (Internet Explorer)
  Ajax support, 648–649
  competition between Netscape and Microsoft and, 34
  CSS standards and, 96
  debugger for JavaScript, 183
  event handlers, 250–251
  navigator object and, 223–224
  popularity of, 8
  pseudo classes and, 102
  XML support, 268
if element, JSTL, 422
if statements
  JavaScript, 156
  Perl, 322
  PHP, 442–443
  Ruby, 575
IIOP (Internet Inter-ORB Protocol), 304
IIS server, 11
IL (Intermediate Language), 481
image, MIME types, 14
images, CSS, 115–116
images, Java. *See* graphics, Java
images, XHTML
  document validation, 49–51
  <img /> tag, 48–49
  overview of, 47–48
  stacking elements and, 247–250
IMAP, PHP support for, 432
<img /> tag, XHTML images, 48–49
<img> tag, Java applets, 687–689
implicit type conversions
  JavaScript, 146–147
  PHP, 438–439
implicit variables
  Perl, 315
  Ruby, 565
implode function, PHP arrays, 448
include construct, PHP, 433–434
increment (++), unary operator, 143, 315
inheritance
  C++, 662
  conflict resolution and, 127
  Java, 670
  property values, 104
  prototype-based, 137
  Ruby, 594–595
inline, CSS levels, 97–99
inline tags, XHTML, 45
input
  file input in Perl, 317, 339–341
  keyboard input in JavaScript, 150–154
  keyboard input in Perl, 317–318
  keyboard input in Ruby, 571–573
<input> tag, XHTML, 69–74
  checkboxes and, 71–72
  controls used with, 69–70
  labels and, 71
  radio buttons and, 73–74
  Submit and Reset buttons created with, 77–78
INSERT clauses, SQL, 527
instances, XML schema, 283–284
InstantRails, 607. *See also* Rails
Integer class
  Ruby, 563
  servlets, 395
integer types
  Perl, 313
  PHP, 435
integrity, Web security issues, 19
interactive applets, Java, 696
Interactive Ruby, 566
interfaces, Java, 670
Intermediate Language (IL), 481
internal DTD, 272
Internationalization and Formatting library, JSTL, 414
Internet
  domain names, 4–6
  IP addresses, 4
  origins of, 2–3

overview of, 3
World Wide Web compared to, 7
Internet Explorer. *See* IE (Internet Explorer)
Internet Inter-ORB Protocol (IIOP), 304
interoperability, Web services and, 304
interpolation, PHP, 436
interpret mode, PHP processor, 433
`intval` function, for explicit type conversion, 439
IP (Internet Protocol), 4
IP addresses
converting domain names to, 5
overview of, 4
IPv6, 4
`irb`, Interactive Ruby, 566
ISO (International Standards Organization)
JavaScript, 134
SGML, 266
SQL, 524–525
text-formatting languages, 34
iterators, Ruby, 595–597

J

J#.NET, 480–481
`JApplet` class, 682, 684
Java, 661–676
applets. *See* applets, Java
binding exceptions to handlers, 671–672
C++ compared with, 662–664
classes, objects, and methods, 666–669
data types and structures, 664–666
event model, 701–705
example of exception handling, 674–675
exception classes, 670–671
exception handlers, 671
exception propagation, 672–673
interfaces, 670
JavaScript compared with, 135
JDBC architecture for database access, 531–532
overview of, 24–25, 661–664
Ruby compared with, 561
servlets. *See* servlets, Java
summary, 675–676
`throws` clause, 673
Java Server Pages. *See* JSP (Java Server Pages)
Java Virtual Machine (JVM), 481
Java Web Services Developer Pack, 305
JavaBeans
components, 480
JSTL and, 414
JavaScript, 133–188
Ajax and, 638
anchors for patterns, 178
array methods, 166–168
array object characteristics, 164–165
array object creation, 163–164
assignment statements, 149
character and character class patterns, 176–178
constructors, 174–175
control expressions, 155–156
`Date` object, 150
dynamic XHTML documents. *See* dynamic documents, XHTML
embedding in XHTML documents, 139–140
errors in scripts, 181–183
event-driven compilation, 136
exercises, 187–188
explicit type conversions, 147–148
functions, 168–169
implicit type conversions, 146–147
Java and, 135
Java applets compared with, 683
keyboard input, 150–154
local variables, 169
loop statements, 159–161
`Math` object, 145
`Number` object, 145–146
numeric operators, 143–145
object creation and modification, 162–163
object-orientation, 137–138
origins of, 134
overview of, 24, 133–134
parameters, 169–172
pattern matching with regular expressions, 175–176
pattern matching with `string`, 179–180
pattern modifiers, 178–179
PHP compared with, 26
primitive types, 141–143
review questions, 185–187
screen output, 150–154
selection statements, 156
`sort` method, 172–173
string catenation operator, 146
string properties and methods, 148–149
summary, 183–185
`switch` statement, 157–159
syntax of, 138–141
`typeof` operator, 149
uses of, 135–136
variable declaration, 143
Web browsers and, 136–137
JavaScript, for XHTML documents
attributes, 196–199
DOM 0, 191–193
DOM 2, 217
DOM tree modification, 225
DOM tree traversal, 224
element access, 193–195
event handler registration, 218–220
event handling, 195–196

event propagation, 217–218
events, 196–199
example with DOM 2, 220–222
execution environment, 190–191
exercises, 227–228
`focus` events, 207–209
handling events from body elements, 199–201
handling events from button elements, 201–206
handling events from text box and password elements, 207
`navigator` object, 222–224
overview of, 136–137
review questions, 226–227
summary, 225–226
tags, 196–199
validating form input, 209–216
`javax.servlet` package, 389
`javax.swing`, 697–701
JDBC, database access, 531–532
JDBC and My SQL, 548–556
example, 553–556
metadata, 551–553
overview of, 548–551
Jet database engine, 530
JIT (Just-In-Time) compilers, 481
`join` method, JavaScript array methods, 166
joins, SQL, 526–527
Joint Photographic Experts Group (JPEG), 47–48
JPEG (Joint Photographic Experts Group), 47–48
JScript, 134. *See also* JavaScript
JScript.NET, 480–481
JSP (Java Server Pages). *See also* servlets, Java
ASP.NET compared with, 492
exercises, 429–430
JSP documents, 413–415
JSP Expression Language, 419–422
JSTL action elements, 422–426
motivation for using, 412–413
overview of, 412
review questions, 428–429
scriptlets, 415–419
server-side computation, 350
summary, 426–427
JSP documents
combining XHTML and JSP documents, 417–419
elements, 416
Expression Language and, 420–422
overview of, 413–415
scriptlets, 415–419
JSP Standard Tag Library. *See* JSTL (JSP Standard Tag Library)
JSTL (JSP Standard Tag Library)
action elements, 422–426
Expression Language and, 419–422
libraries, 414
Just-In-Time (JIT) compiler, 481
JVM (Java Virtual Machine), 481

K

`key` function, PHP arrays, 450
keyboard input
JavaScript, 150–154
Perl, 317–318
Ruby, 571–573
`keys` operator, Perl hashes, 330
keyword parameters, 589
keywords, property values, 103

L

labels
`<input>` tag, 71
Java Swing components, 697
row or column labels, 63–65
`<th>` tag, 61
LAMP (Linux, Apache, and MySQL), 350
`last` operator, Perl, 323
LaTeX, 21
layout documents, Rails, 633–635
layout managers, Java, 698
`left` property, stacking elements and, 247
length, property values, 103
Lerdorf, Rasmus, 432
less than (<), for Ruby subclasses, 594
`letter-spacing` property, CSS, 110
levels, CSS, 97–98
`<li>` tag, list items, 55
`lighter` values, `font-weight` property, 106
line breaks, XHTML document structure, 40–41
line input operator, Perl, 318–319
lines, Java graphics, 692
`line-through` value, `text-decoration` property, 109
`<link>` tag, external style sheets and, 98
links
CGI to XHTML document, 351–354
overview of, 52–55
targets within documents, 54–55
uses of, 55
Web documents connected by, 6
Linux, Apache, and MySQL (LAMP), 350
LISP-based functional languages, 293
list properties, CSS, 110–114
`ListControl` class, ASP.NET, 506
lists, Perl, 324–325
list assignment, 325
list literals, 324
lists, Ruby, 580–583
lists, XHTML
definition lists, 59–60
ordered lists, 56–59

overview of, 55
unordered lists, 55–56
list-style-image property, CSS, 111
list-style-type property, CSS, 110–111
LiveScript, 134
load events, DOM 0, 199–201
local declarations, DTDs, 285
local variables
JavaScript, 169
Perl, 332
PHP, 457
Ruby, 586–587
loop statements
JavaScript, 159–161
Perl, 321–323
PHP, 443–444
Ruby, 575–578

M

m operator, pattern matching in Perl, 336
mail, 10
mailto, protocols running on top of TCP/IP, 5
Managed C++.NET, 480–481
margin property, CSS, 121–123
margins, CSS box model, 118, 121–123
markup
markup languages, 21
server-side controls, 507
match method, JavaScript, 179–180
Math module, Ruby, 565
Math object, JavaScript, 145
Matsumoto, Yukihiro, 26, 562
maxlength attribute, <input> tag, 70
meta element, XHTML documents, 46–47
metacharacters, pattern matching and, 176
metadata, 551–553
meta-markup languages, 23, 266
method attribute
<form> tag, 355
XHTML forms, 69
method definition, Ruby, 585
method header, Ruby, 585
methods
C#, 486–487
C++, 668
Cookie class, 400
DOM, 192–193
HttpServlet class, 389
JApplet class, 684
Java, 666–669
Java strings, 665
JavaScript, 137
methods, Ruby
built-in methods for arrays and lists, 580–583
local variables, 586–587
overview of, 585–586
parameters, 587–590
string methods, 566–570
Microsoft, competition with Netscape, 34
Microsoft Access, 530
Microsoft Intermediate Language (MSIL), 481
MIME (Multipurpose Internet Mail Extensions)
experimental document types, 14–15
overview of, 13
type specifications, 13–14
mixin, Ruby modules, 595
mod_perl, servlets compared with, 388
model, in MVC architecture, 606
Model-View-Controller (MVC), 27, 606–607
modules, Ruby, 595
modulus (%) , binary operator, 143, 315
Mosaic, 7, 34
mouse clicks, reacting to, 253–254
mouse cursor
dynamic changes to fonts, 242–243
help boxes and, 246
locating, 250–252
mousedown events, 253, 257
MouseEvent interface, 250
mouseover events, 242–243
mouseup events, 253, 257
Move It button, moving elements, 236
MSIL (Microsoft Intermediate Language), 481
multiplication (*), binary operator, 143, 315
Multipurpose Internet Mail Extensions. *See* MIME (Multipurpose Internet Mail Extensions)
MVC (Model-View-Controller), 27, 606–607
MySQL. *See also* SQL (Structured Query Language)
connecting to and selecting a database, 540
as database management system, 350
JDBC and, 548–556
overview of, 532–534
Perl and, 535–537
PHP and, 539–547
Rails and, 622–623
requesting MySQL operations from PHP, 540–541
MySQL Administrator, 534
mysql_connect function, PHP, 540
mysql_query function, PHP, 540–541

N

name attribute, HTML vs. XHTML, 89
named data types, XML schemas, 285
names servers, 5
names/naming conventions
arrays, 324
domain names, 4–6
Java, 663

JavaScript, 139
Ruby variables, 564
XML, 269
XML entities, 271
namespaces, XML, 279–281
National Center for Supercomputer Applications (NCSA), 7
National Science Foundation (NSF), 3
navigator object, Web browsers in XHTML documents, 222–224
NCSA (National Center for Supercomputer Applications), 7
negate (-), unary operator, 143
nested lists, 58–59
nested tags, as alternative to attributes in XML, 270
.NET Framework
ASP.NET and, 350
background of, 480
CLR and, 481–483
.NET languages and, 480–481
overview of, 479–480
Web services and, 304, 513
Netscape, competition with Microsoft, 34
News, Web servers supporting, 10
next function, PHP, 450
next operator, Perl, 323
next statements, Ruby, 577
none value, text-decoration property, 109
nonrepudiation, Web security issues, 19
NOT (!) operator
JavaScript, 155
Perl, 320
Ruby, 574
NoteCards, Xerox, 6
NSF (National Science Foundation), 3
null type, JavaScript, 142
NULL type, PHP, 435
Number object, JavaScript, 145–146
numbers
JavaScript, 142
property values, 103
Numeric class, Ruby, 563
numeric literals (types)
JavaScript, 142
Perl, 313
Ruby, 563–564
numeric operators
JavaScript, 143–145
Perl, 315, 320
Ruby, 565–566

O

Object, JavaScript, 138
Object Remote Procedure Call (ORPC), 304
<object> tag, Java applets, 687–689
object-oriented languages
C#, 486
JavaScript, 137–138
method calls, 668
Ruby, 26, 565
object-relational mapping (ORM), 606
objects, Java, 666–669
objects, JavaScript. *See also* Arrays, JavaScript
creating and modifying, 162–163
Math object, 145
Number object, 145–146
octothorpe (#), 101
ODBC (Open Database Connectivity), 530
ODBC driver manager, 530
 tag, ordered lists, 56–57
onclick attribute, image tags, 247
Open Database Connectivity (ODBC), 530
open function, files, 339
operations, Ruby, 562–563
operators, C#, 484
operators, JavaScript
assignment, 149
binary and unary, 143
comparison, 172–173
numeric, 143–145
precedence and associativity rules for, 156
relational, 155
string, 146
operators, Perl
assignment, 317–319
binary and unary, 315
built-in list functions, 328
control statements, 323
numeric, 315
pattern matching, 336–339
precedence and associativity rules for, 315, 321
relational, 320
string, 316
operators, PHP
arithmetic, 437
assignment, 440
Boolean, 442
relational, 442
string, 437–438
operators, Ruby
assignment, 565, 574
numeric, 565–566
pattern matching, 598
precedence and associativity rules for, 574
relational operators, 573
string catenation, 567
<option> tag, XHTML, 74
OR (||) operator
JavaScript, 155
Perl, 320
Ruby, 574

ordered lists
  sequencing values for, 112–113
  XHTML, 56–59
ORM (object-relational mapping), 606
ORPC (Object Remote Procedure Call), 304
output
  from ASP.NET documents, 494
  Java, 663
  JavaScript screen output, 150–154
  Perl file output, 317, 339–341
  Perl screen output, 318–319
  PHP, 440–441
  Ruby screen output, 571
ovals, Java graphics, 695
`overline` value, `text-decoration` property, 109

P

<p> tag, XHTML text markup, 38–40
package scope, Java, 667
packages, Java, 667
padding, CSS box model, 118, 121–123
page-level events, ASP.NET, 503
PageMill, Adobe, 22
pages, units of information on Web, 6
`paintComponent` method, Java applets, 685–687
paragraphs, XHTML text markup, 38–40
<param> tag, Java applets, 689
parameter entities, 276
parameters
  Java applets, 689–691
  Perl, 333–334
  PHP, 454–455
parameters, JavaScript, 169–172
  actual parameters, 169
  formal parameters, 170
  passing, 170–172
parameters, Ruby, 587–590
  asterisk parameters, 588
  formal and actual, 587
  keyword parameters, 589
parentheses (`( )`), forcing operator precedence with, 145
parsable character data (PCDATA), 274
`parseInt` method, 395
parsers, XML, 277
partial paths, URLs, 13
pass-by-reference parameters
  C#, 486
  PHP, 455
pass-by-value parameters
  C#, 486
  JavaScript, 170
  PHP, 454
Passport, 513
passwords, handling events from password elements, 207, 212–216
paths, URL, 12–13
pattern matching, JavaScript
  anchors for patterns, 178
  character and character class patterns, 176–178
  pattern modifiers, 178–179
  regular expressions, 175–176
  with `string`, 179–180
pattern matching, Perl
  overview of, 336–337
  remembering matches, 338
  substitute operator, 338
  transliterate operator, 339
pattern matching, PHP, 457–460
pattern matching, Ruby
  overview of, 597–599
  remembering matches, 599–600
  substitutions, 600
pattern modifiers, 178–179
PCDATA (parsable character data), 274
PCRE (Perl-Compatible Regular Expression) library, 457
percentages, property values, 103
Perl, 311–348. *See also* CGI (Common Gateway Interface)
  array example, 328–329
  arrays, 324–326
  assignment operators, 317
  built-in arrays, 327
  built-in list functions, 327–328
  `CGI.pm` module, 356–359
  control expressions, 319–321
  `cookie` function, 378–381
  DBI/DBD architecture for database access, 530–531
  example program, 341–343
  exercises, 346–348
  file input and output, 339–341
  `foreach` statement, 326
  function example, 334–336
  functions, 331–332
  hashes, 329–330
  keyboard input, 317–318
  list literals, 324
  local variables, 332
  loop statements, 321–323
  numeric literals, 313
  numeric operators, 315
  origins of, 312
  overview of, 25, 311–312
  parameters, 333–334
  pattern matching, 336–337
  references, 330–331
  remembering matches, 338
  review questions, 345–346

Ruby compared with, 562
scalar variables, 312–315
screen output, 318–319
selection statements, 321–323
`sort` function, 334
string literals, 313
string operators and string functions, 316
substitute operator, 338
summary, 343–345
transliterate operator, 339
Perl and MySQL, 535–537
DBI module, 535–537
example, 537–538
overview of, 535
`perl` command, 319
Perl-Compatible Regular Expression (PCRE) library, 457
Personal Home Page Tools, 432
PHP, 431–478
Ajax response document written in, 645
arithmetic operators and expressions, 437
array access, 447, 449–451
array creation, 445–447
array functions, 447–453
array sorting, 451–453
assignment operators, 440
Boolean operators, 442
Boolean type, 436
control statements, 442
cookies, 469–471
database access, 531
double type, 435
embedded in XHTML documents, 433
example, 444–445
exercises, 475–477
file management, 466–469
form handling, 460–466
functions, 453–454
`htmlspecialchars` function, 539
integer type, 435
loop statements, 443–444
`mysql_connect` function, 540–541
`mysql_query` function, 540–541
origins of, 432
output, 440–441
overview of, 26, 431–433
parameters of functions, 454–455
pattern matching, 457–460
relational operators, 442
review questions, 474–475
scalar type conversions, 438–439
selection statements, 442–443
server-side programming or scripting languages, 350
session tracking, 471–472
string operators, 437–438
string type, 436
summary, 472–473
syntax, 433–434
variable lifetime, 457
variable scope, 455–457
variables, 435
PHP and MySQL, 539–547
connecting to MySQL and selecting a database, 540
example, 541–547
overview of, 539
requesting MySQL operations, 540–541
special characters, 539–540
plain, form action buttons, 69, 78
`plain`, MIME types, 14
plain text, XML documents, 268
plug-ins, 14, 22–23
plus (+), symbolic quantifier in pattern matching, 178
plus (+), unary operator, 143
PNG (Portable Network Graphics), 48
pointers, C#, 483, 488
polygons, Java graphics, 694–695
polymorphism, JavaScript support for, 137
`pop` function, Perl, 327
`pop` method
JavaScript, 167
Ruby, 580–581
POP3, 432
Portable Network Graphics (PNG), 48
positioning elements
absolute positioning, 231–234
overview of, 230–231
relative positioning, 234–236
static positioning, 236
POSIX, pattern matching with, 457
`POST` method
form methods, 69
HTTP requests, 16, 515
query strings and, 354–356
Rails and, 616
Web services and, 304
postback, ASP.NET, 499
postfix operators, JavaScript, 143
pound sign (#), Perl comments, 317
`pre` tag, XHTML document structure, 41
precedence rules
JavaScript, 144, 156
Perl, 315, 321
Ruby, 565–566, 574
prefix operators, JavaScript, 143
`preg_match` function, PHP pattern matching, 458
`preg_split` function, PHP pattern matching, 458
presentations, with XSLT, 294–302
primary keys, relational databases, 522
primitives
C#, 483–484

Java, 664
XML schema, 284
primitives, JavaScript
numeric and string literals, 142
other primitive types, 142–143
overview of, 137
types of, 141
print function
Perl, 318
PHP, 440–441
Ruby, 571
privacy, Web security issues, 19
program structure, C#, 488–489
programming code, server-side controls, 507
programming languages
BNF (Backus-Naur form) and, 273
functional-style, 293
languages, 312
meta-markup, 23, 266
scripting, 350, 432
prompt method, JavaScript, 152
properties
C#, 487–488
DOM, 193
JavaScript, 137–138
property value forms, CSS, 103–104
property values, conflict resolution and, 127
Prototype, 607
prototype-based inheritance, 137
proxy servers, 10
pseudo classes, CSS, 102–103
public-key encryption, 19
push function, Perl, 327
push method, JavaScript, 167
puts method, Ruby, 571
Python, 350

Q

query strings, 354–356
question mark (?), symbolic quantifier in pattern matching, 178
quoted attribute values, HTML vs. XHTML, 88
qw function, Perl, 328

R

radio buttons
click events, 201–206
control type value, 73
DOM addressing and, 194–195
<input> tag, 73–74
Java Swing components, 697
JSTL, 425
Rails, 605–637
application setup, 615–616
controller and view for form application, 617–621
database construction, 622–627
document requests, 607–608
dynamic documents, 612–615
exercises, 636–637
form handling, 615
layouts, 633–635
overview of, 27, 605–607
review questions, 636
static documents, 608–612
summary, 635–636
testing application against database, 627–633
Rails, with Ajax
controller method, 655–657
form document, 650–652
overview of, 649–650
Rails triggering Ajax, 652–655
Rails documents
dynamic, 612–615
requests, 607–608
static, 608–612
Random class, ASP.NET, 494–495
random number generator, 494–495
raw XML documents, 289–291
receiver phase, Ajax, 646–648
rectangles, Java graphics, 692–694
Reenskaug, Trygve, 606
Reference types, CTS, 482
references
Java, 662, 664–665
Perl, 330–331
registration
connecting events to event handlers, 196
DOM 0, 218
DOM 2, 218–220
properties vs. attributes in, 206
regular expressions
pattern matching in JavaScript, 175–176
pattern matching in PHP, 457–458
Relational Database library, JSTL, 414
relational databases, 522–524
example, 522–524
overview of, 522
relational operators
JavaScript, 155
Perl, 320
PHP, 442
Ruby, 573
relative value, positioning elements, 234–236
Remote Procedure Call (RPC), 304
removeEventListener method, DOM 2, 219
render blocks, ASP.NET documents, 493
request phase, Ajax, 643–645
requests, HTTP, 15–17
header fields, 16–17
methods, 15
running CGI programs and, 351
requests, Rails document, 607–608

reserved words
C#, 484
Expression Language, 420
JavaScript, 139
PHP, 434
SQL, 525
Reset, action buttons, 69–70, 77–78
resources, units of information on Web, 6
response document, Ajax, 645–646
response output for controls, ASP.NET, 508
responses, HTTP, 17–18, 353–354
`Response.Write` method, ASP.NET, 508
`reverse` method, JavaScript arrays, 166
RFC 2616 (HTTP standard), 15
RGB (red, green, blue)
CSS property values, 103
Java colors, 696
Rivest, Ron, 20
root elements, XML, 269
RoR (Ruby on Rails), 27, 606. *See also* Rails
`rows` attribute, `<frameset>` tag, 82
`rowspan` attribute, XHTML tables, 63–65
RPC (Remote Procedure Call), 304
RSA public key algorithm, 20
Ruby, 561–603
access control, 592–594
array example, 583
arrays, 578–579
assignment statements, 564–565
built-in methods for arrays and lists, 580–583
classes, 590–592
code blocks and iterators, 595–597
control expressions, 573–574
exercises, 602–603
`for-in` statements, 579–580
hashes, 583–585
inheritance, 594–595
keyboard input, 571–573
local variables, 586–587
loop statements, 575–578
methods, 585–586
numeric literals, 563–564
numeric operators, 565–566
origins of, 562
overview of, 26–27, 561–562
parameters, 587–590
pattern matching and, 597–599
Rails and, 606
remembering matches, 599–600
review questions, 601–602
scalar types and operations, 562–563
screen output, 571
selection statements, 575–578
string literals, 563–564
string methods, 566–570
substitutions, 600
summary, 601
variables, 564–565
Ruby on Rails. *See* RoR (Ruby on Rails)
rules
CSS rule lists, 99
XHTML tables, 61

S

SAX (Simple API for XML), 303
scalar types, PHP
conversions, 438–439
overview of, 434
scalar types, Ruby
numeric and string literals, 563–564
numeric operators, 565–566
overview of, 562–563
string methods, 566–570
variables and assignment statements, 564–565
scalar variables, Perl, 312–315
overview of, 314–315
references, 330–331
symbol for, 312
types of values in, 312–313
`schema`, as root element in XML schemas, 282
schemas, XML. *See* XML schemas
Scheme, 293
scope, variable
JavaScript, 169
PHP, 455–457
screen output
Java, 663
JavaScript, 150–154
Perl, 318–319
Ruby, 571
`<script>` tag, 138
scripting languages
JavaScript, 24
PHP, 432
types of, 350
scriptlets, JSP, 415–419
scripts. *See also* JavaScript
client-side and server-side, 134
errors in, 181–183
scriptlets, 415–419
`scrolling` attribute, `<frame>` tag, 84
sections, XHTML tables, 68–69
security, Web. *See* Web security
`SELECT` clauses, SQL
joins, 526
types of, 525
`<select>` tag, 74–76
selection statements
JavaScript, 156
Perl, 321–323
PHP, 442–443
Ruby, 575–578

selectors, CSS
class selectors, 100–101
generic selectors, 101
id selectors, 101
pseudo classes, 102–103
simple selectors, 99
universal selectors, 101–102
sequence elements, 286
sequence value types, CSS lists, 112–114
server controls
coding control elements, 507
control events, 504–505
document life cycle, 499–503
example, 508–510
HTML controls, 497–499
methods for specifying, 507
overview of, 497
page-level events, 503
response output for controls, 508
validation controls, 510–513
Web controls, 505–507
server root, Web server directories, 9
ServerChange events, ASP.NET, 503
ServerClick events, ASP.NET, 503
server-side scripting languages
controls. *See* server controls
JavaScript, 134
PHP, 432
servlet containers
multiple access to servlets in, 395
overview of, 388
Tomcat servlet container, 392
servlet engines, 388
Servlet interface, 389
ServletRequest interface, 389
ServletResponse interface, 389
servlets, Java. *See also* JSP (Java Server Pages)
ballot form example, 401–407
cookies, 399–401
details, 389–392
exercises, 429–430
JSP documents and, 412–413
multiple access to servlets in servlet containers, 395
overview of, 25, 387–388
review questions, 428–429
session tracking, 407–412
summary, 426–427
survey example, 392–398
session objects, 408
session tracking, 407–412
overview of, 408–409
PHP, 471–472
vote counter example, 409–412
sessions
browser/server interaction, 377
overview of, 399, 469
setcookie function, 470–471
setInterval method, 254
setter methods, Ruby classes, 593
setTimeout method, moving elements slowly, 254
settype function, for explicit type conversion, 439
SGML (Standard Generalized Markup Language), 23
DOCTYPE command, 37–38
HTML based on, 34
as meta-markup language, 266
XML as simplified version of, 267
sh, UNIX shell, 312
Shamir, Adi, 20
shift function, Perl, 327
shift method
JavaScript array methods, 167
Ruby, 580
shortcuts, Perl functions, 357
shorthand, CSS fonts, 106108
SHOW TABLES command, MySQL, 533–534
Simple API for XML (SAX), 303
simple data type, XML schemas, 284, 285–286
Simple Object Access Protocol (SOAP), 305, 515
simple selectors, CSS, 99
single quotes (')
Perl delimiters, 313–314
PHP delimiters, 436
Ruby delimiters, 564
size attribute, <input> tag, 70
sizes, font
CSS, 105
XHTML, 41–42
slice method, JavaScript array methods, 166
Smalltalk, 26, 606
SOAP (Simple Object Access Protocol), 305, 515
software development framework, 607
sort function
Perl, 327–328, 334
PHP arrays, 451–453
sort method
JavaScript, 172–173
Ruby, 580–581
source, link, 52
 tags, CSS, 125–126
special characters
PHP, 539
PHP and My SQL, 539–540
URLs and, 12
widgets and, 355
split function, Perl, 327
split method
JavaScript, 180
Ruby, 598
sprintf function, Rails, 617–618
SQL (Structured Query Language). *See also* MySQL

CREATE TABLE clauses, 528–529
DELETE clauses, 528
DROP clauses, 528
INSERT clauses, 527
joins, 526–527
overview of, 524–525
SELECT clauses, 525
UPDATE clauses, 527–528
SQL injection attacks, 628
src attribute, <frame> tag, 83
srm.conf, 11
stacking elements, 247–250
Standard Generalized Markup Language. *See* SGML (Standard Generalized Markup Language)
standards
CSS, 96
DOM, 191
JavaScript, 134
SGML, 266
SQL, 524–525
text-formatting languages, 34
XML, 266, 279
static documents, Rails, 608–612
static value, positioning elements, 236
status codes, HTTP responses, 17–18
STDERR, Perl filehandles, 317–318
STDIN, Perl filehandles, 317–318
STDOUT, Perl filehandles, 317–318
Stein, Lincoln, 356
string catenation operator
JavaScript, 146
Perl, 316
PHP, 437
Ruby, 567
String class, Ruby, 566–570
string classes, Java, 665
string functions
Perl, 316–317
PHP, 438
string literals
JavaScript, 142
Perl, 313–314
PHP, 436
Ruby, 563–564
String methods, JavaScript, 148–149, 179–180
string methods, Ruby, 566–570
String objects, Ruby, 563
string operators
Perl, 316, 320
PHP, 437–438
string repetition operator (x), Perl, 316
strings, converting arrays to, 448
 tag, 44
structs, C#, 487
Structured Query Language. *See* SQL (Structured Query Language)
structures
C# data structure, 486–487
DOM trees, 191–192
Java data structures, 664–666
XHTML documents. *See* document structure, XHTML
XML documents, 271–272
style sheets. *See* CSS (cascading style sheets)
<style> tag, 98
styles, CSS fonts, 106
sub method, Ruby, 600
<sub> tag, XHTML, 45
Submit button
action buttons, 69–70, 77–78
interacting with Web server via forms, 351
XHTML forms, 69
subscripts, array, 325
substitute operator, Perl, 338
substitutions, Ruby, 600
subtraction (-), binary operator, 143, 315
Sun Microsystems, 24
SunONE servers, 8
survey example, CGI, 365–377
Swing package (javax.swing), 691, 697–701
switch statement
C#, 485–486
JavaScript, 157–159
PHP, 443
symbols, Ruby, 589
syntax
Expression Language, 419
HTML vs. XHTML, 88–89
JavaScript, 138–141
JSP documents, 413
PHP, 433–434
XHTML, 36–37
XML, 268–270
XSLT, 293
System, FCL (Framework Class Libraries), 488
System.object, .NET types deriving from, 482

T

<table> tags, XHTML, 61–63
tables, CSS, 115–116
tables, XHTML, 61–69
align and valign attributes, 65–66
cellpadding and cellspacing attributes, 67–68
rowspan and colspan attributes, 63–65
sections, 68–69
<table> tags, 61–63
tags
events and, 196–199
syntax of, 36
XHTML, 21

target node, DOM 2, 217
targets
  within documents, 54–55
  link, 52
tbody, table sections, 68
TCP/IP (Transmission Control Protocol/Internet Protocol), 3
<td> tag, data tags, 61
telnet
  determining IP address of a Web site, 5
  HTTP requests, 16
template text, JSP documents, 413
template-driven model, XSLT, 294
templates, Rails, 607, 610
Tex, markup language, 21
text
  alignment, 116–117
  dynamic changes to content of text fields, 244–246
  float property, 115–116
  Java paintComponent method, 685–687
  letter-spacing property, 110
  over background image, 125
  text processing languages, 312
  text-decoration property, 109–110
  text-formatting languages, 34
  XML documents as plain text, 268
text alignment, CSS, 116–117
text boxes
  handling events from text box elements, 207
  Java Swing components, 697
  labeling, 71
  mouse events in, 250
  uses of, 70
text markup, XHTML, 38–47
  block quotations, 42–43
  character entities, 45–46
  fonts styles and sizes, 43–45
  headings, 41–42
  horizontal rules, 46
  line breaks, 40–41
  meta element, 46–47
  overview of, 38
  paragraphs, 38–40
  whitespace, 41
text processing languages, 312
text, MIME types, 14
text-align property, CSS, 115
<textarea> tag, XHTML, 76–77
text-decoration property, 109–110
text-formatting languages, 34
tfoot, table sections, 68–69
<th> tag, labels, 61
thead, table sections, 68
Thompson, Henry S., 288
three-tier systems, client/server architectures, 530
throw clause, C++, 673
Throwable class, Java, 670
throws clause, Java, 673
tiling, background images, 125
times, iterator method in Ruby, 595
Tobin, Richard, 288
Tomcat servlet container, 392
Tools-Options-Advanced window, 127
top property, stacking elements and, 247
toString method, JavaScript array methods, 146, 167
tr (transliterate) operator, 339
<tr> tag, row tag, 61
transliterate operator, Perl, 339
Transmission Control Protocol/Internet Protocol (TCP/IP), 3
trivial types, JavaScript, 141
TRUE, PHP Boolean types, 436
try clause, Java exception handling, 671–672
two-tier systems, client/server architectures, 529–530
type attribute
  <input> tag, 69
  <style> tag, 99
type conversion
  explicit, 147–148
  implicit, 146–147
  PHP, 438–439
typeof operator, JavaScript, 149

U

UDDI (Universal Description, Discovery, and Integration Service), 305, 516
 tag, unordered lists, 55–56
unary operators
  JavaScript, 143
  Perl, 315
unbound variables, PHP, 435
unchecked exceptions, Java, 673
undef value, Perl variables, 314
undefined type, JavaScript, 143
uniform resource identifiers (URIs)
  user-defined namespaces and, 281
  XML namespaces and, 279
uniform resource locators. *See* URLs (uniform resource locators)
Universal Description, Discovery, and Integration Service (UDDI), 305, 516
universal selectors, CSS, 101–102
UNIX
  Rails for, 608
  shell, 312
unless statement
  Perl, 322
  Ruby, 575
unload events, DOM 0, 199–201
unordered lists, XHTML, 55–56
unset function, PHP, 447

unshift function, Perl, 327
unshift method
  JavaScript, 167
  Ruby, 580–581
until statements
  Perl, 322
  Ruby, 577
UPDATE clauses, SQL, 527–528
URIs (uniform resource identifiers)
  user-defined namespaces and, 281
  XML namespaces and, 279
URLs (uniform resource locators)
  format, 11–12
  overview of, 11
  paths, 12–13
  property values, 103
  query strings and, 355–356
  server/browser communication and, 8
Usenet, 5
user-defined classes, Java, 668
user-defined data types, XML schema, 284
user-defined functions, PHP, 453–457
UTC (Coordinated Universal Time), 613

V

valid, XML documents, 277
validating parsers, XML, 277
validation
  form input, 209–216
  server controls, 510–513
  XHTML documents, 49–51
  XML documents against XML schema, 284
  XML schemas, 288–289
valign attribute, XHTML tables, 65–66
value property, accessing element content, 244
Value types, CTS, 482
value types, JavaScript, 138
values, XML attribute, 275
values operator, Perl hashes, 330
variable scope
  JavaScript, 169
  PHP, 455–457
variables, Expression Language, 419–420
variables, JavaScript
  declaring, 143
  local variables, 169
  referring to as objects, 137
variables, Perl
  arrays. *See* arrays, Perl
  hashes. *See* hashes, Perl
  local, 332
  scalar. *See* scalar variables, Perl
  types of, 312
variables, PHP
  $ (dollar sign) for, 434
  lifetime of, 457
  overview of, 435
  scope, 455–457
variables, Ruby
  implicit, 565
  local, 586–587
  naming conventions, 564
  overview of, 564–565
VB (Visual Basic), 505
VB.NET, 480–481
versions, XHTML, 34–35
video, MIME types, 14
view
  in MVC architecture, 606
  Rails form, 617–621
View Source, PHP, 433
ViewState, ASP.NET, 500–502
virtual document trees, Web servers and, 10
virtual hosts, 10
viruses, 20
visibility property, elements, 239–240
Visual Basic (VB), 505
Visual Studio, 489

W

W3C (World Wide Web Consortium)
  CSS standard, 96
  DOM standard, 191
  on frames, 82
  WSDL standard for Web services, 305
  XML namespace standard, 279
  XML schemas, 281
  XML standard, 266
Wall, Larry, 312
Web
  exercises, 32
  Internet compared with, 7
  origins of, 6
  review questions, 30–32
  summary, 28–30
Web applications
  Ajax and, 638
  Rails and, 606
Web browsers
  Ajax and, 648–649
  cookies and, 401
  CSS and, 96
  event handlers, 250–251
  JavaScript and, 136–137
  navigator object, 222–224
  overview of, 7–8
  Rails requests, 610
  sessions of browser/server interaction, 377
  XML and, 268
Web controls, ASP.NET, 505–507
Web palette, of colors, 114–115
Web programming tools

Ajax, 27–28
Java, 24–25
JavaScript, 24
overview of, 20–21
Perl, 25
PHP, 26
plug-ins and filters, 22–23
Rails, 27
Ruby, 26–27
XHTML, 21–22
XML, 23
Web security
overview of, 18–20
SQL injection attacks, 628
Web servers
Apache server, 10–11
CGI location on, 351–352
characteristics of, 9–10
IIS server, 11
operation of, 8–9
overview of, 8
sessions of browser/server interaction, 377
Web services, 513–516
advertising, 516
constructing, 513–516
overview of, 304–305, 513
Web Services Definition Language (WSDL), 305, 515
weight, CSS, 106, 127
well formed XML, 269
what-you-see-is-what-you-get (WYSIWYG), 21–22
`WHERE` clause, SQL, 525
`while` statements
JavaScript, 159–161
Perl, 322
PHP, 443
whitespace, XHTML document structure, 41
widgets, 354–355
HTML controls compared with, 505–506
in Swing package, 697
XHTML tags for, 69
`width` attribute, XHTML images, 48
Window methods, 254
`Window` object, JavaScript, 151, 190
word processors, plug-ins for, 22–23
World Wide Web. *See* Web
World Wide Web Consortium. *See* W3C (World Wide Web Consortium)
worms, Web security and, 20
wrapper class, Java, 664
wrapper objects
`Integer` class, 395
JavaScript, 141
`write` method, JavaScript, 151–152
WSDL (Web Services Definition Language), 305, 515
WYSIWYG (what-you-see-is-what-you-get), 21–22

X

Xerces-J, 302
XHTML (eXtensible Hypertext Markup Language)
action buttons, 77–78
`align` and `valign` attributes, 65–66
block quotations, 43–44
`cellpadding` and `cellspacing` attributes, 67–68
character entities, 45–46
definition lists, 59–60
document validation, 49–51
dynamic documents. *See* dynamic documents, XHTML
exercises, 92–94
fonts styles and sizes, 44–45
form example, 78–81
forms, 69
frames, 83–87
framesets, 82–83
headings, 41–42
horizontal rules, 46
HTML compared with, 35–36
HTML controls and, 498
image formats, 47–48
`<img />` tag, 48–49
`<input>` tag, 69–74
JavaScript for. *See* JavaScript, for XHTML documents
line breaks, 40–41
link uses, 55
links, 52–54
`meta` element, 46–47
`<object>` tag for referencing applets in, 687–689
ordered lists, 56–59
origins of, 34
overview of, 21
paragraphs, 38–40
review questions, 90–92
`rowspan` and `colspan` attributes, 63–65
`<select>` tag, 74–76
standard document structure, 37–38
summary, 89–90
syntactic differences between HTML and XHTML, 88–89
syntax of, 36–37
table sections, 68–69
`<table>` tags, 61–63
targets within documents, 54–55
text markup, 38
`<textarea>` tag, 76–77
transforming XML into XHTML, 293
unordered lists, 55–56
versions of, 34–35
whitespace, 41
XHTML tags used with XML documents, 279
XML compared with, 23

XML syntax and, 269
XHTML documents
applet parameters, 689–691
applets providing processing and interactivity for, 682–683
ASP.NET embedding programming code into, 25
block quotations, 43–44
body, 38
CGI linking to, 351–354
`CGI.pm` module for boilerplate functions in, 358–359
computations, 24–25, 682–683
converters, 22–23
dynamic documents. *See* dynamic documents, XHTML
Expression Language and, 421–422
HTTP sending XHTML documents, 12
interactive applets, 696
JavaScript embedded in, 24
JavaScript for. *See* JavaScript, for XHTML documents
JSP documents combined with, 417–419
PHP processor outputting, 433
problems with embedding in XHTML documents, 139–140
structure. *See* document structure, XHTML
targets, 54–55
tools for creating, 21–22
validation of, 49–51
XSLT and, 296–302
XHTML editors, 21–22
XML (eXtensible Markup Language), 265–310
Ajax and, 639
declaring attributes, 275–276
declaring elements, 273–274
declaring entities, 276
displaying raw XML documents, 289–291
displaying XML documents with CSS, 291–292
document structure, 271–272
document type definitions, 272–273
exercises, 309–310
internal and external DTDs, 277–279
namespaces, 279–281
overview of, 23, 265–268
review questions, 307–309
sample DTD, 276–277
summary, 305–307
syntax of, 268–270
XML applications, 267
XML declaration, 37, 268–269
XML documents
defined, 268
displaying raw XML documents, 289–291
displaying with CSS, 291–292
document type definitions, 272–276
DTDs inside of, 277–279
formatting options, 293
structure of, 271–272
valid, 277
validating against XML schema, 284
XML namespace and, 279–281
XML Processing library in JSTL, 414
XSLT and, 295–302
XML namespace, 279–281
XML parsers, 277, 302
XML Path Language (XPath), 293, 296
XML Processing library, JSTL, 414
XML processors
DOM approach, 303–304
inability to analyze DTDs, 281
overview of, 302
purposes of, 302–303
SAX approach, 303
XML Schema Validator. *See* `xsv` utility
XML schemas
complex types, 286–288
data types, 284–285
defining a schema, 282–283
defining schema instances, 283–284
overview of, 281–282
simple types, 285–286
validating schema instances, 288–289
XML syntax and, 268
XML syntax
JSP documents and, 413
XHTML and, 269
XML schemas and, 268
XML-DEV user group, 303
`XMLHttpRequest` object, 643–644
`xml-stylesheet`, 292
XPath (XML Path Language), 293, 296
`xsd`, 285
`xsl`, 295
XSL (eXtensible Stylesheet Language), 293
XSL documents, 293
XSL Transformations. *See* XSLT (XSL Transformations)
XSL-FO (XSL Formatting Objects), 293
XSLT (XSL Transformations), 295
formatting objects in, 295
overview of, 293–294
for presentations, 294–302
style sheet technology, 291
transforming XML into XHTML, 293
XHTML documents, 296–297
XML documents, 295–296
`xsv` utility
validating schema instances, 288–289
validating XML documents against XML schema, 284

INTERNET / WORLD WIDE WEB PROGRAMMING

Programming the World Wide Web is a student-oriented introduction to the fundamental concepts underlying Web programming (client-side and server-side development). This edition includes three new chapters on Ruby, Rails, and Ajax, as well as many updates throughout the book.

Dr. Robert W. Sebesta is an associate professor of computer science at the University of Colorado at Colorado Springs. He received his Ph.D. in computer science from Pennsylvania State University.

TOPICS COVERED INCLUDE:

Fundamentals

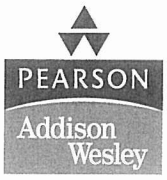
- A Brief Introduction to the Internet
- Web Browsers
- Web Servers
- URL
- MIME
- HTTP
- The Web Programmer's Toolbox

Client-Side Technologies

- HTML and XHTML
- Cascading Style Sheets
- JavaScript™
- XML and XSLT
- Applets

Server-Side Technologies

- Perl™/CGI
- Servlets and Java Server Pages
- PHP
- ASP.NET Using C#
- Database Access through the Web
- Ruby
- Rails
- Ajax



Visit www.aw-bc.com/computing for more information about Addison-Wesley computing books. To order any of our products, contact our customer service department at (800) 824-7799 or (201) 767-5021 outside of the U.S., or visit your campus bookstore.

Addison-Wesley Computing
Leading Authors • Quality Products

ISBN-13: 978-0-321-48969-2
ISBN-10: 0-321-48969-1